

**VIETNAM–KOREA UNIVERSITY OF INFORMATION AND
COMMUNICATION TECHNOLOGY
FACULTY OF COMPUTER ENGINEERING AND
ELECTRONICS**



GRADUATION PROJECT

PROJECT TITLE:

**AN AI-ENABLED AUTOMATIC PARKING
MANAGEMENT SYSTEM USING LICENSE PLATE
RECOGNITION AND RFID**

Student Full Name:	<i>Truong Quoc Tu</i>
Class:	<i>21CE2</i>
Major:	<i>Computer Engineering Technology</i>
Specialization:	<i>Computer Engineering Technology</i>
Supervisor:	<i>M.Sc. Nguyen Thi Huyen Trang</i>

Da Nang – 1/2026

**VIETNAM–KOREA UNIVERSITY OF INFORMATION AND
COMMUNICATION TECHNOLOGY
FACULTY OF COMPUTER ENGINEERING AND
ELECTRONICS**



GRADUATION PROJECT

PROJECT TITLE:

**AN AI-ENABLED AUTOMATIC PARKING
MANAGEMENT SYSTEM USING LICENSE PLATE
RECOGNITION AND RFID**

Student Full Name:	<i>Truong Quoc Tu</i>
Class:	<i>21CE2</i>
Major:	<i>Computer Engineering Technology</i>
Specialization:	<i>Computer Engineering Technology</i>
Supervisor:	<i>M.Sc. Nguyen Thi Huyen</i>

Da Nang – 1/2026

ACKNOWLEDGEMENTS

First and foremost, I would like to express my sincere gratitude to M.Sc. Nguyen Thi Huyen Trang for her support and assistance throughout my study and the implementation of this graduation project. Moreover, during the internship period, the lecturer devotedly instructed and guided me in both theoretical knowledge and practical skills, as well as problem-solving approaches. The lecturer was always a source of encouragement and support, helping me successfully complete the project implementation phase. I would also like to sincerely thank all lecturers of the Faculty for their dedication and guidance, and for creating favorable conditions for me throughout my study at the university. I would like to express my heartfelt thanks to my family, who have always stood by my side and encouraged me during the most difficult times. Thank you all very much

[illegible]

Supervisor

ii

ABSTRACT

Project Title: “An AI-Enabled Automatic Parking Management System Using License Plate Recognition And RFID”

The project "An AI-Enabled Automatic Parking Management System Using License Plate Recognition And RFID" aims to develop and test an intelligent parking management model using Radio Frequency Identification (RFID) technology. The system employs RFID tags attached to vehicles and RFID readers installed at entry and exit gates, connected to a microcontroller (Arduino) for data processing. When a user scans a tag, the system verifies the tag code against the stored database; if valid, the gate automatically opens, while information is displayed on an LCD screen and supported by alert signals such as LEDs or a buzzer. The system helps minimize human errors, save time, and enhance the efficiency of vehicle access management. Through the simulation process, the project evaluates the feasibility of the model in practice and establishes a foundation for the development of modern intelligent parking systems.

TABLE OF CONTENTS

ACKNOWLEDGEMENTS	i
SUPERVISOR’S COMMENTS.....	ii
ABSTRACT	iii
TABLE OF CONTENTS	iv
LIST OF ABBREVIATIONS.....	vii
LIST OF FIGURES.....	viii
LIST OF TABLES.....	x
INTRODUCTION	1
CHAPTER 1: THEORETICAL BASIS.....	4
1.1 Chapter Introduction	4
1.2 IoT System	4
1.2.1. Introduction.....	4
1.2.2. Basic Architecture of an IoT System	5
1.3. Software Used in the System	6
1.3.1. Proteus Simulation Software	6
1.3.2. Overview of Arduino Software.....	7
1.4. Strengths of the System Compared to Existing Products.....	13
1.4.1. Existing Smart Parking Projects	13
1.4.2. Outstanding Strengths of the Project	14
1.5. Factors Affecting the System	15
1.5.1. Distance and Orientation Between RFID Tag and Reader	15
1.5.2. Electromagnetic Interference (EMI)	15
1.5.3. Microcontroller Processing Speed	15
1.6. Main Components Used in the Circuit.....	16
1.6.1. ESP32 Module.	16
1.6.2. RFID RC522 Module.....	20
1.6.3. RFID Card (MIFARE Tag).....	24
1.6.4. Servo Motor SG90	25
1.6.5. Buzzer (5V chip buzzer).	27

1.6.6. Display Block.	28
1.6.7. 5V Relay Module.....	31
1.6.8. LM2596 Step-Down (Buck) Module.....	32
1.6.9. Raspberry pi 3.....	34
1.7 Conclusion of the Chapter.....	38
CHAPTER 2: SYSTEM ANALYSIS AND DESIGN.....	39
2.1. Functions and Selection of Main Components for Each Block	39
2.1.1. Identification Block.	39
2.1.2. Display Block	39
2.1.3. Central Processing Block.....	39
2.1.4. Gate Control Block	39
2.1.5. Warning Block.....	40
2.1.6. Power Block.....	40
2.1.7. Relay Block.....	40
2.1.8. Camera & Image Processing Block	40
2.2. Hardware Design for the System.	41
2.2.1. Design Objectives	41
2.2.2. Overall System Block Diagram	42
2.2.3. System Block Diagram	42
2.2.3. Functions and Operating Principles of Each Block	43
2.1.4. Circuit Schematic Design	47
2.1.5. PCB Circuit.....	48
2.2. Analyze and design software for the system.....	49
2.2.1. Program Algorithm Flowchart.....	49
2.2.2. Program Algorithm Explanation::	49
2.3. Chapter Conclusion.....	52
CHAPTER 3: SYSTEM IMPLEMENTATION AND EVALUATION.....	54
3.1. Chapter Introduction	54
3.2. Hardware implementation and performance results.....	54
3.3. Results	55

3.3.1. Printed Circuit Board Results	55
3.3.2. Results of Implementing the Parking Management System	55
3.3.3. Results of Implementing Entry and Exit History for the Parking System.....	56
3.3.4. Warning for Invalid Card Access	57
3.4. System Evaluation.....	59
CONCLUSION AND FUTURE DEVELOPMENT DIRECTIONS OF THE PROJECT	60
1. Conclusion.....	60
2. Limitations	60
3. Future Development Directions	61
4 Conclusion.....	62
REFERENCES	xi

LIST OF ABBREVIATIONS

I2C: Inter-Integrated Circuit

SPI: Serial Peripheral Interface

UART: Universal Asynchronous Receiver/Transmitter

PWM: Pulse Width Modulation

LCD: Liquid Crystal Display

IOT: Internet of thing

LED: Light Emitting Diode

IP: Internet Protocol

JTAG: Joint Test Action Group

SWD: Serial Wire Debug

CAN: Controller Area Network

GPIO: General Purpose Input/Output

ADC: Analog-to-Digital Converter

DAC: Digital-to-Analog Converter

BLE: Bluetooth Low Energy

ESP-IDF: Espressif IoT Development Framework

IDE: Integrated Development Environment

MQTT: Message Queuing Telemetry Transport

HTTP: HyperText Transfer Protocol

LIST OF FIGURES

Figure 1: Traffic congestion	1
Figure 1.1: Illustration of IoT application.	5
Figure 1.2: Four Basic Components of an IoT System	6
Figure 1.3: Proteus Software	6
Figure 1.4: Installing Arduino IDE	8
Figure 1.5: Instructions for installing Arduino software	8
Figure 1.6: Arduino IDE Interface	9
Figure 1.7: Downloading the NodeMCU Package.....	10
Figure 1.8: Board Manager.....	11
Figure 1.9: Board Selection.....	11
Figure 1.10: Port Selection.....	12
Figure 1.11: Actual Image of ESP32.....	18
Figure 1.12: ESP32 Pinout Diagram	20
Figure 1.13: RFID RC522 Module.....	20
Figure 1.14: Pinout Diagram of the RFID RC522 Module	21
Figure 1.15: RFID Card.....	24
Figure 1.16: SG90 Servo Motor	25
Figure 1.17: Buzzer module	27
Figure 1.18: 16x2 LCD Display	29
Figure 1.19: I2C LCD 16x2 Module	30
Figure 1.20: 5V Relay Module	31
Figure 1.21: LM2596 Step-Down Module	33
Figure 1.22: Raspberry Pi 3.....	35
Figure 1.23: Pinout diagram of Raspberry Pi 3.....	35
Figure 1.24: ICON Wedcam Q-20	37
Figure 2.1: Overall Block Diagram	42
Figure 2.2: System diagram when the vehicle enters	42

Figure 2.3: System diagram when the vehicle exits	43
Figure 2.4: System Schematic Diagram	47
Figure 2.5: Wiring Diagram of the Circuit	48
Figure 2.6: 2D Image of the Circuit	48
Figure 2.7: Algorithm Flowchart for Vehicle Entry	49
Figure 2.8: Algorithm Flowchart for Vehicle Exit	49
Figure 3.2: Website Interface and Displayed Information	55
Figure 3.3: Entry and Exit History	56

LIST OF TABLES

Table 1.1: Icon Functions	9
Table 1.2: Esp32 Technical Specifications	18
Table 1.3: Technical Specifications Of Rfid Rc522 Module	22
Table 1.4: Pin Connections Of Rfid Rc522 Module	22
Table 1.5: Technical Specifications Of The Rfid Card	24
Table 1.6: Technical Specifications Of Sg90 Servo Motor.....	26
Table 1.5: Sg90 Servo Motor Pinout.....	26
Table 1.6 Buzzer Classification.....	27
Table 1.7: Buzzer Pinout	28
Table 1.8: 16x2 Lcd Specifications	29
Table 1.9: Specifications Of Lm2596 Step-Down Module.....	33
Table 1.10: Specifications	36
Table 3.1: Components Used For The Circuit Board.....	54

INTRODUCTION

1. Reason for Choosing the Project

In this chapter, I will provide an overview of the simulation of an RFID-based parking system using IoT technology and the objectives of the topic.

Currently, in large urban areas, the congestion of vehicles and the increasing demand for parking lead to challenges in managing parking lots efficiently and safely. Traditional management methods using paper tickets and manual checks not only consume time but are also prone to errors, fraud, and congestion at entry and exit points, especially during peak hours. This significantly affects both user experience and the operational efficiency of parking lots.

In response to this situation, applying advanced technology to parking management is essential. Among modern solutions, Radio Frequency Identification (RFID) technology stands out with its contactless recognition capability, fast processing speed, and high accuracy. Using RFID tags instead of paper tickets enables the system to automatically control vehicle entry and exit, enhance transparency, save manpower, and reduce processing time..



Figure 1: Traffic congestion

However, deploying a practical RFID system requires appropriate investment and technical implementation. Therefore, building a simulation model is an important step to verify the operational feasibility, efficiency, and practical applicability of this technology in parking management. This is the reason why the group chose to conduct the project “Simulation of an RFID-Based Parking System,” aiming to explore, design, and test a simple yet effective automation model that can be applied and expanded in future real-world projects.

2. Project Objectives

- Develop a simulation model of an automatic parking system using RFID technology to control vehicle entry and exit.
- Study and apply RFID technology combined with a microcontroller (Arduino) in practice.
- Automate the parking process to save time, reduce errors, and enhance management efficiency.
- Design a simple display interface such as an LCD and alert signals using LEDs/Buzzer to indicate the status of tags and gates.
- Evaluate the feasibility and operational performance of the system before real-world deployment.

3. Implementation Content

a. Theoretical Research

- Study the operating principles of RFID technology and related components (tags, readers, microcontrollers, etc.).
- Learn programming and simulation on the Arduino platform.

b. Selection of Devices and Tools

- Choose suitable microcontroller (ESP32), RFID reader (RC522), RFID tags, servo motor, LCD, buzzer, LED, etc.
- Use Arduino IDE software for programming and simulation.

c. System Schematic Design

- Connect components according to an appropriate diagram to simulate the vehicle entry and recognition process.
- Design circuits so that gates can automatically open/close based on tag authentication status.

d. System Control Programming

- Write Arduino programs to read data from RFID tags, process information, and control output devices.
- Integrate information display on LCD and alerts via LED or buzzer.

e. Simulation and Testing

- Simulate system operation on actual hardware or supporting software such as Proteus.
- Test cases including: valid tags, invalid tags, and multiple consecutive scans.

f. Result Evaluation and Development Recommendations

- Evaluate operational efficiency, advantages, and limitations of the system.
- Propose improvements such as storing parking data, connecting to a computer, or remote management systems.

4. Conclusion

The introduction presented an overview of the project, including the reasons for choosing the topic, its urgency, objectives, scope, and research methods. This demonstrates that applying RFID technology to parking management is an inevitable trend in the digital era, helping to improve efficiency, save time, and minimize errors in vehicle management. At the same time, this chapter clearly outlined the research content and initially determined the approach for implementing the simulation model of the system. The contents presented in this chapter serve as an important foundation for developing subsequent chapters, which will delve deeper into system design, programming, and testing.

CHAPTER 1: THEORETICAL BASIS

1.1 Chapter Introduction

In the process of building any technical system, mastering the theoretical foundation and understanding the function and operating principles of each component is a prerequisite to ensure accuracy and efficiency. This chapter presents the foundational knowledge related to RFID technology-the core technology of the automatic parking system-while also providing detailed introductions to the hardware and software components used in the project, such as RFID tags, RC522 readers, ESP32, servo motor, LCD display, LEDs, and buzzer. Understanding the role and operation of each component will facilitate a smoother and more accurate process in system design, programming, and simulation in the subsequent chapters.

1.2 IoT System

1.2.1. Introduction

IoT (Internet of Things) is a modern technology trend that enables physical devices to connect with each other and with the Internet to automatically collect, exchange, and process data. With the rapid development of sensors, wireless networks, and cloud computing, IoT is increasingly applied in various fields such as smart agriculture, smart homes, environmental monitoring, remote healthcare, and especially in intelligent transportation and smart cities.

In the field of parking management, IoT helps connect sensors, cameras, RFID recognition systems, and control software into a synchronized and automated system. As a result, users can remotely monitor vehicle entry and exit, track real-time data, store parking information, and even make online payments. The system can be easily expanded and connected to a server or centralized management platform to serve large-scale management needs. Integrating IoT into parking systems not only enhances user experience but also optimizes operations, reduces labor costs, and contributes to the development of smart cities.

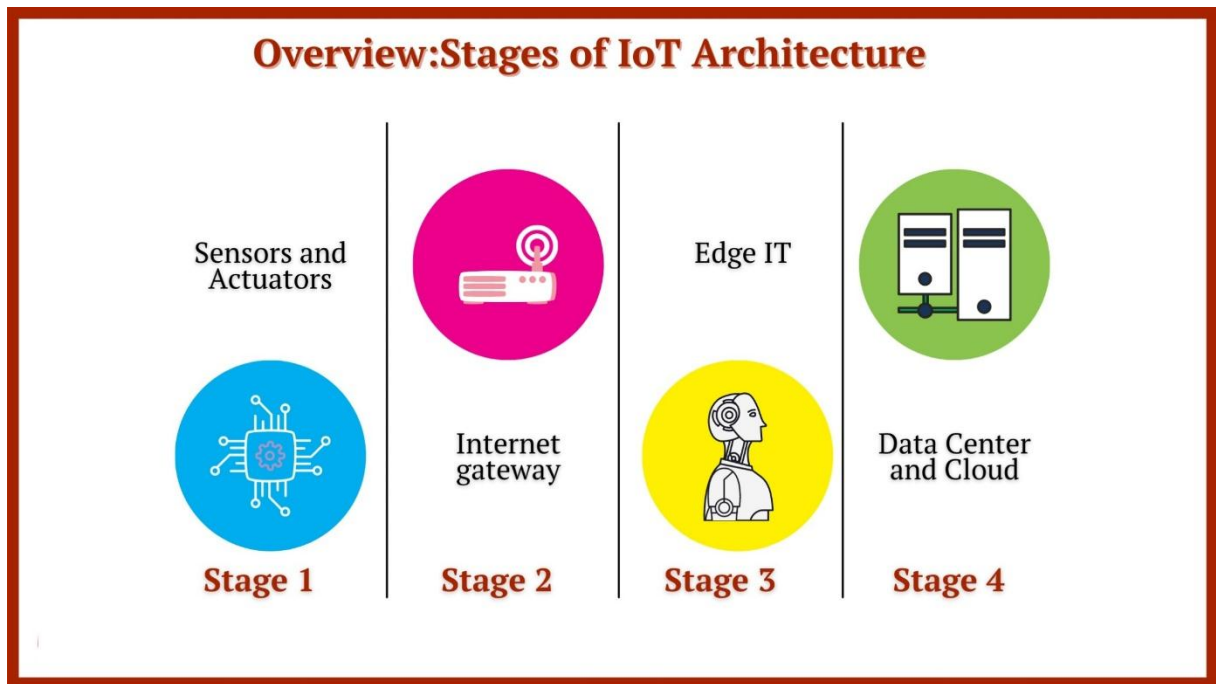


Figure 1.1: Illustration of IoT application.

1.2.2. Basic Architecture of an IoT System

The architecture of IoT consists of four main components: Things, Gateways, Network and Cloud Infrastructure, and finally the Services Layer (Services-Creation and Solutions Layers).

Things: Today, countless items exist in daily life, on farms, in homes, or on users' mobile devices. IoT solutions enable smart devices to filter, connect, and manage data from local agricultural objects, while non-smart devices can be connected via gateways. As a result, devices and items can perform their functions to ensure proper management of agricultural objects.

Gateways: Gateways act as an intermediary zone, allowing these available utilities to connect securely and manageably with the cloud. A gateway can be a physical device or software used to link the cloud with controllers, sensors, and smart devices.

Network and Cloud Infrastructure: The Internet is a global system consisting of multiple interconnected IP networks linked to computer systems. This network infrastructure includes routers, gateways, aggregators, duplicate tags, and many other devices that can control data traffic and are also connected to telecommunication and cable networks deployed by service providers.

Services Layer (Services-Creation and Solutions Layers): Applications created by technology companies or even users allow effective utilization of IoT products and

maximize their value..

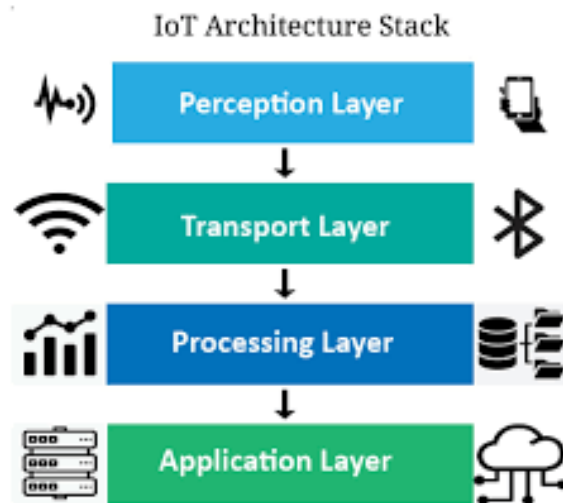


Figure 1.2: Four Basic Components of an IoT System

1.3. Software Used in the System

1.3.1. Proteus Simulation Software

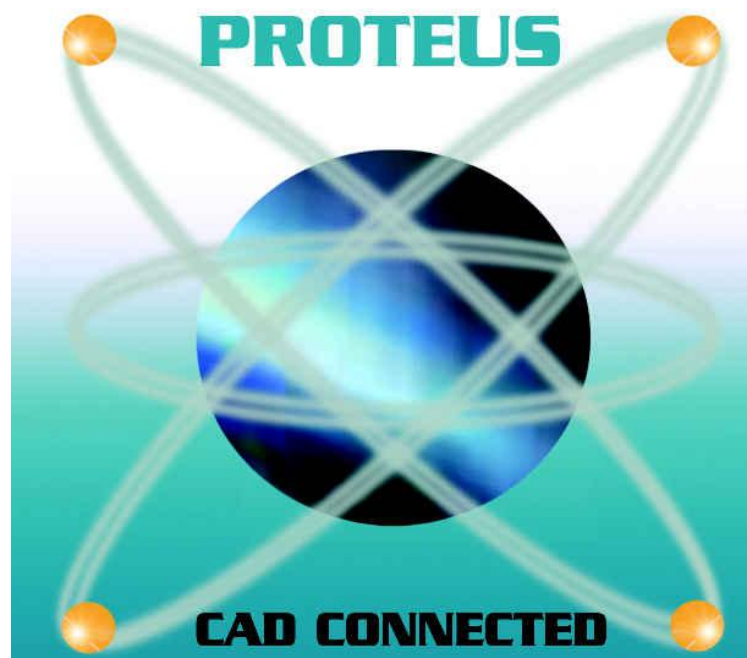


Figure 1.3: Proteus Software

Proteus Design Suite is a software application developed by Proteus Technologies. It provides an interactive electronic design environment (IDE) for developing analog and digital circuits and testing their functionality in the laboratory or in the field quickly and cost-effectively.

Proteus allows users to see, hear, feel, and interact with any analog circuit, complex devices such as sensors, embedded systems, and robots, virtual reality

environments, and information models of buildings and infrastructure. By enabling engineers to rapidly create design prototypes before they are physically built, Proteus minimizes risks during early testing in the development process while shortening schedules when implementing changes at the end of the cycle.

Key features of Proteus:

- a Capable of simulating most microcontroller drivers.
- b Easily select objects and set parameters for them.
- c Export Netlist files compatible with common PCB design software.
- d Generate component statistics for circuits.
- e ISIS integrates multiple tools to manage large circuits, supporting designs with thousands of components for professional circuit development.
- f Hierarchical design capabilities.
- g Automatic component numbering

1.3.2. Overview of Arduino Software

Arduino IDE (Integrated Development Environment) is the official programming software provided by Arduino, used for developing embedded applications on the Arduino platform. It is an open-source development environment that supports editing, compiling, and uploading programs (sketches) directly to microcontrollers such as Arduino Uno, Mega, Nano, and others.

Arduino IDE uses a programming language based on C/C++, simplified with powerful libraries to interface with sensors, actuators, LCDs, RFID modules, and more. Its interface is user-friendly and easy to use for beginners, featuring syntax checking, code compilation, and a Serial Monitor to observe real-time data transmission and reception. Additionally, Arduino IDE allows the installation of additional libraries and expansion through the Board Manager to support various types of microcontrollers.

Thanks to these features, Arduino IDE has become a popular tool for students, engineers, and hobbyists interested in DIY IoT devices, robotics, and automated control applications.

Installing Arduino IDE

Step 1: Download the Arduino software directly from the official website:

Link: <https://www.arduino.cc/en/Main/Software>

Step 2: Install and run the downloaded file.

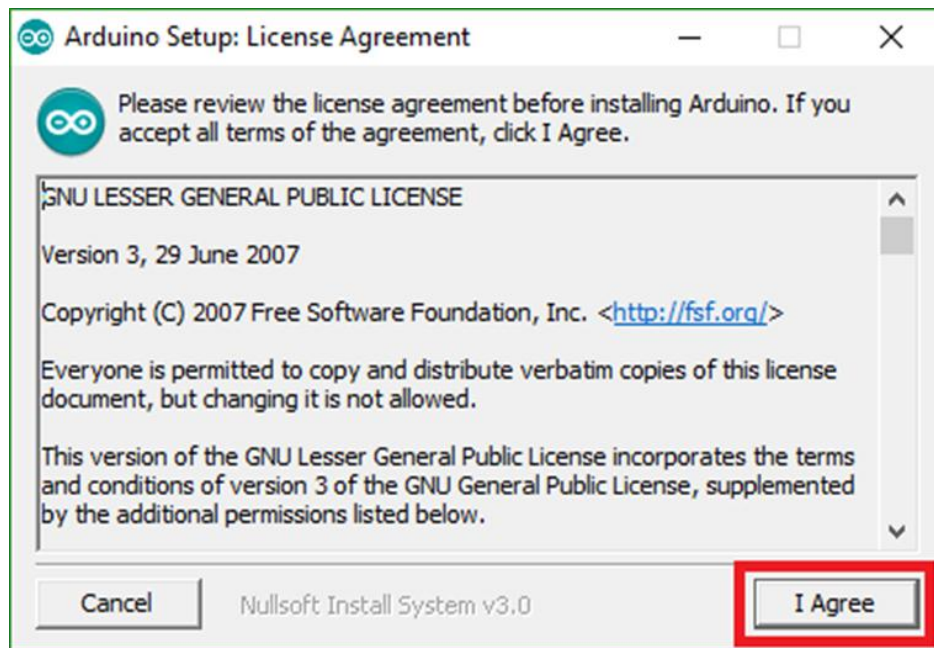


Figure 1.4: Installing Arduino IDE

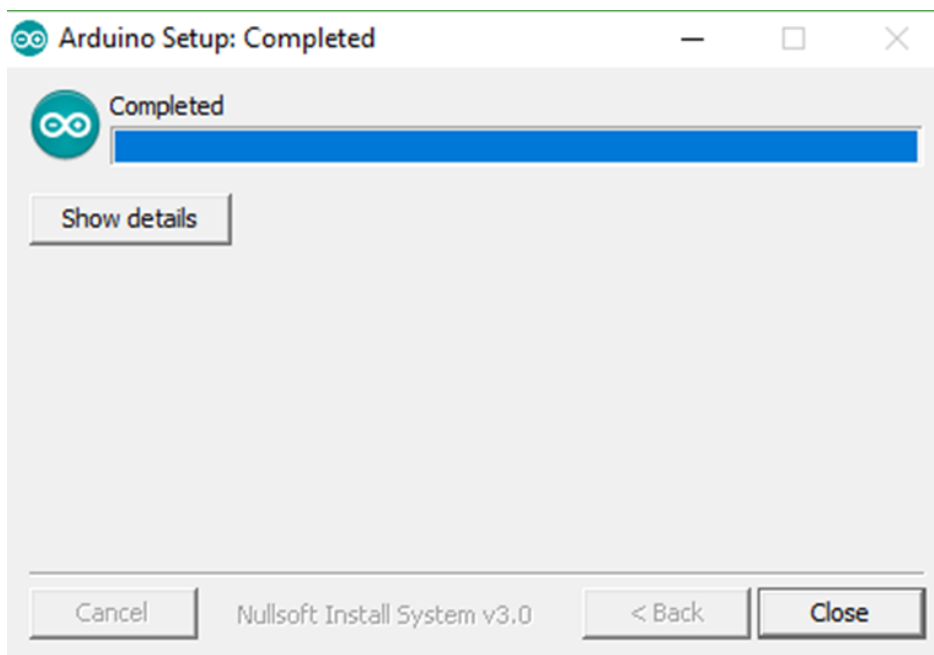


Figure 1.5: Instructions for installing Arduino software

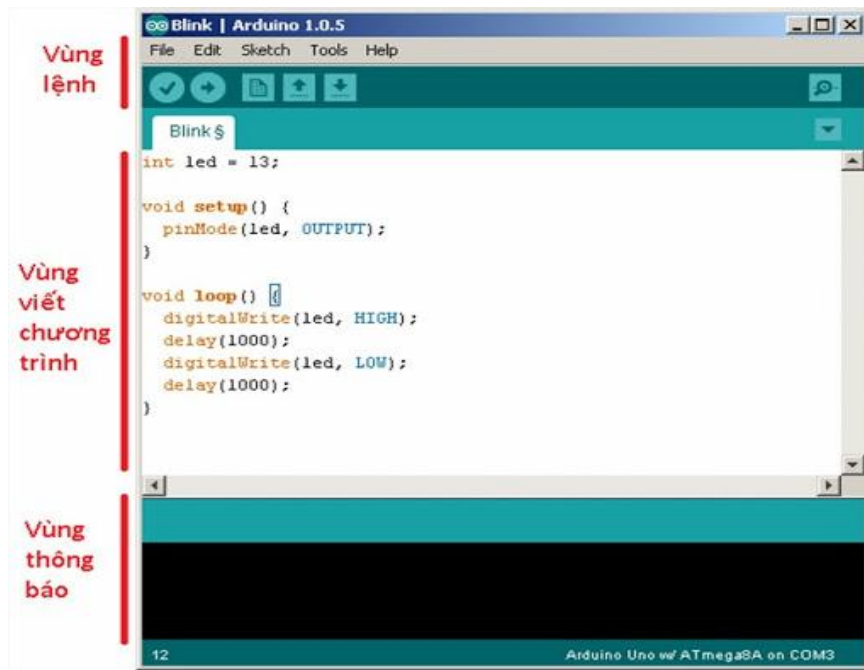


Figure 1.6: Arduino IDE Interface

Command Area: Includes menu commands (File, Edit, Sketch, Tools, Help). Below are icons that allow quick access to commonly used IDE functions.

Icon Functions

Table 1.1: Icon Functions

	Verify	Check for errors and compile code
	Upload	Translate and upload code to a pre-installed board
	New	Create a new sketch
	Open	Open an existing sketch
	Save	Save sketch
	Serial Monitor	Mở serial monitor.

Program Writing Area:

Setup Void: This section is used to configure the Arduino IDE program. It is used to set the data transmission speed, define pins as input or output, and other initial settings.

Loop Void: Used to write commands in the program for the Arduino circuit to perform desired tasks.

Message Area (Debugging): Displays messages from the IDE.

Arduino IDE Configuration:

Step 1: Download the NodeMCU package into Arduino IDE:

From the main screen, select File, Preferences, and add the link below to the Additional Boards Manager URLs section.

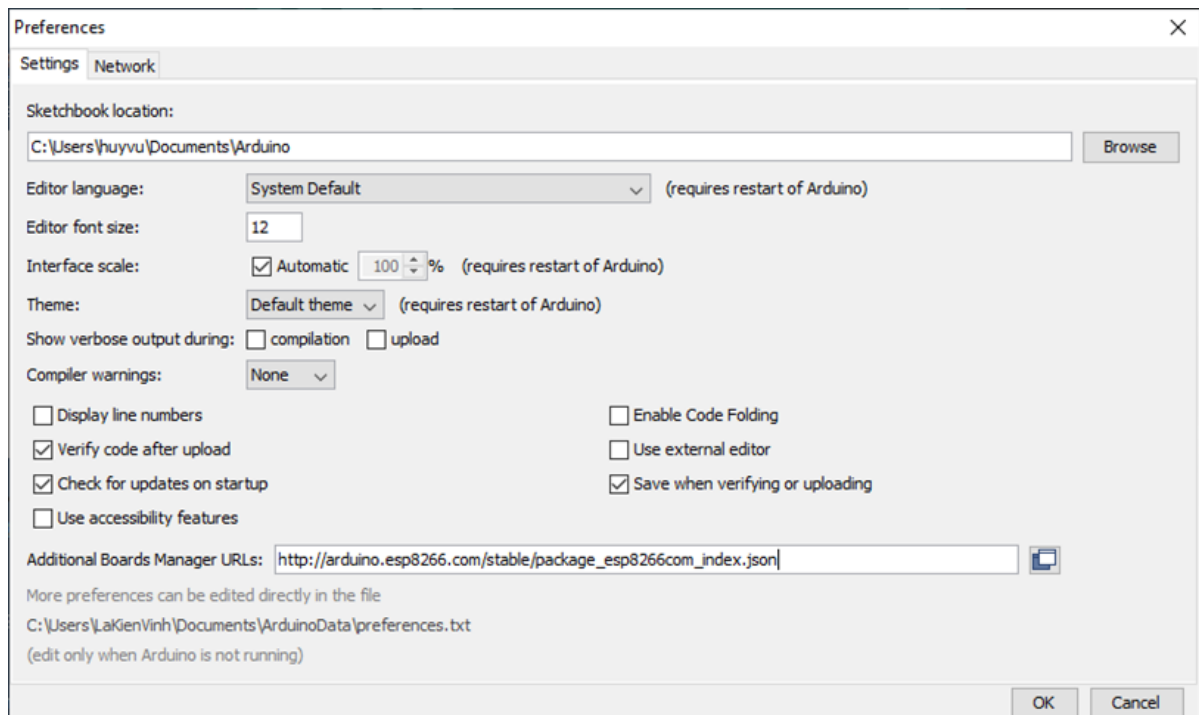


Figure 1.7: Downloading the NodeMCU Package

Step 2: Download the supporting libraries.

From the main screen, select Tools → Board → Board Manager. In the search bar of the Board Manager dialog, enter ESP32, then select Install to download and install the library

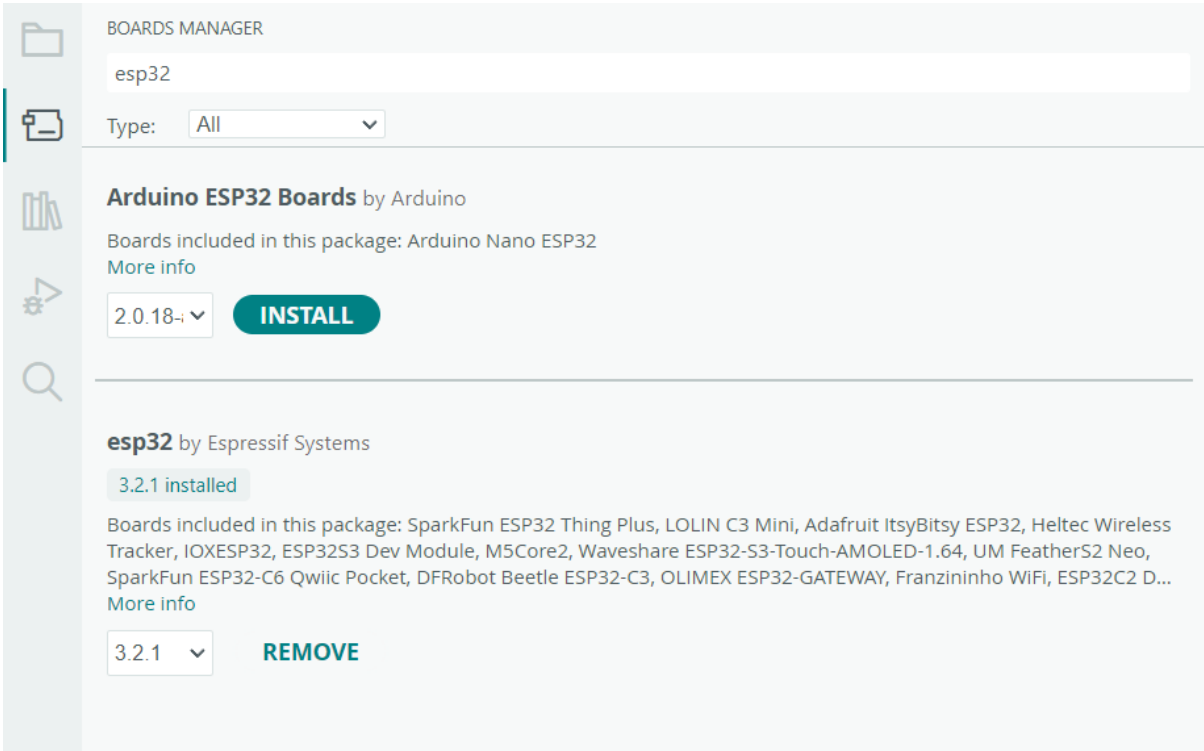


Figure 1.8: Board Manager

Step 3: Select Tools → Board → ESP32 Boards → ESP32 Dev Module

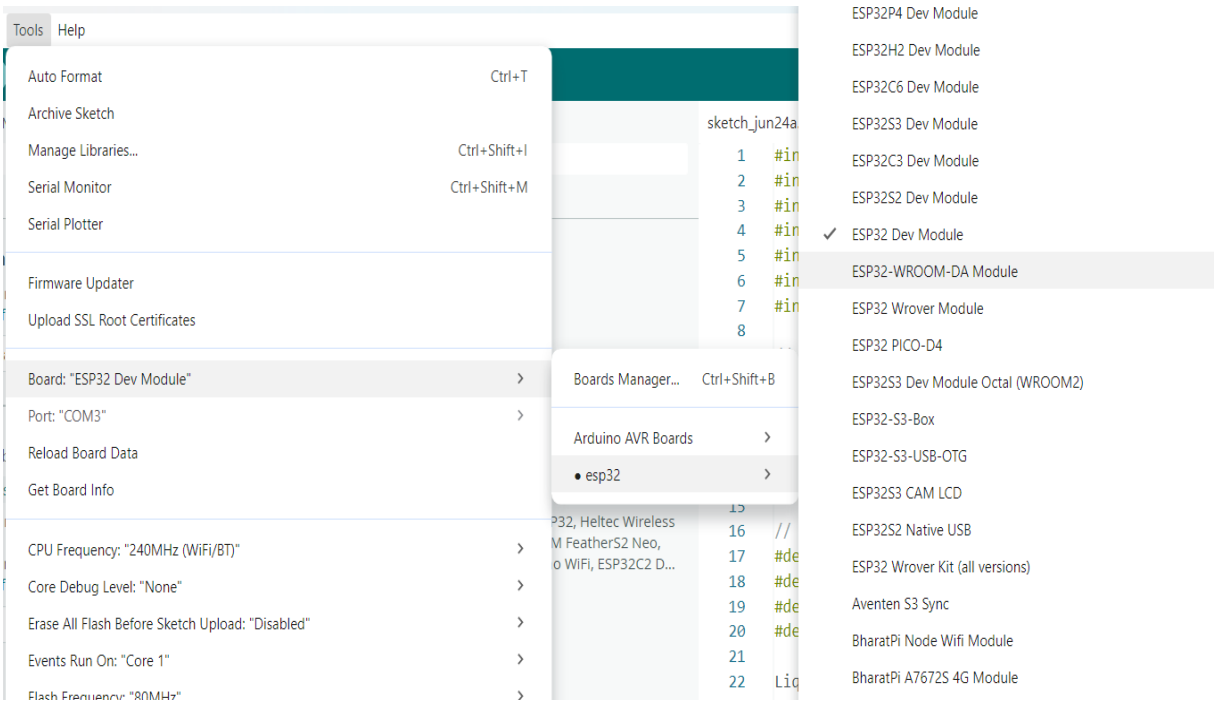


Figure 1.9: Board Selection

Step 4: From the menu, select Tools, Port, then choose the COM port to connect to the computer

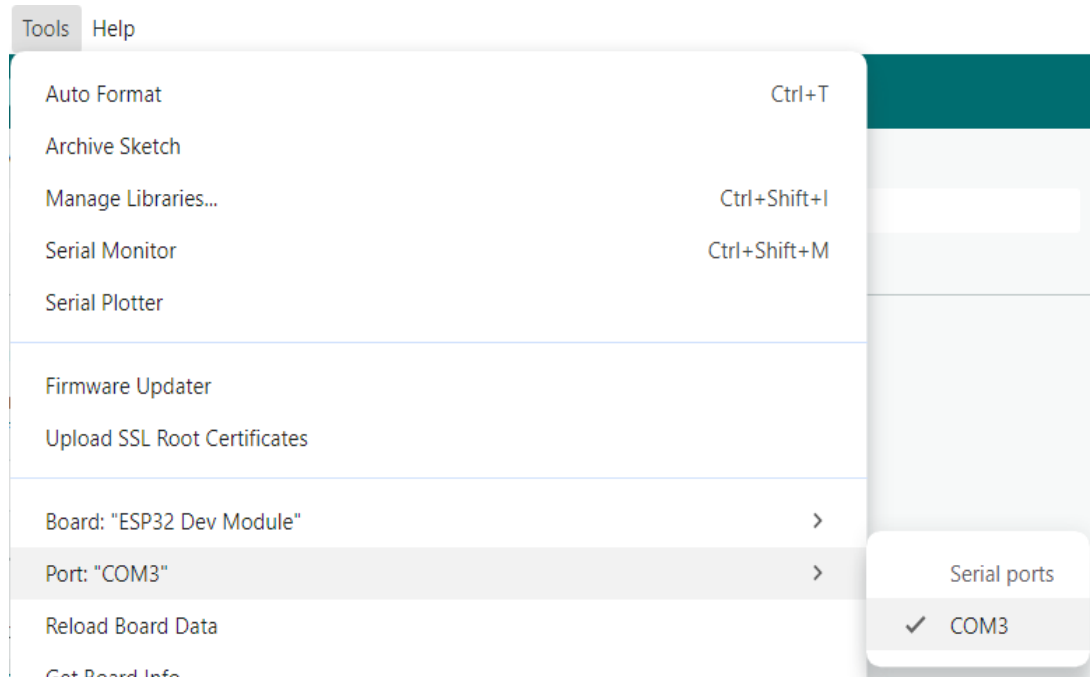


Figure 1.10: Port Selection

Common Applications of Arduino:

Arduino is widely applied in many fields due to its simplicity, low cost, and flexible expandability. Some typical applications include:

- + Automation & IoT: Controlling lighting systems, automatic irrigation, monitoring temperature and humidity via the Internet.
- + Robotics & Autonomous Vehicles: Driving line-following robots, obstacle avoidance, and remote-controlled vehicles.
- + Security Systems: Unlocking via RFID, fingerprint scanning, facial recognition combined with cameras.
- + Measurement Devices: Measuring temperature, distance, gas concentration, and power consumption.
- + Education & Research: A popular platform for students to learn microcontroller control and embedded circuit design.
- + Medical & Environmental Applications: Heart rate monitors, air quality monitors, plant monitoring systems.
 - Advantages:
- + Easy Accessibility: Arduino IDE is user-friendly, code is close to C/C++, with many sample examples.
- + Low Cost: Arduino boards and expansion modules are inexpensive, suitable for

students and research purposes.

- + Open Source & Large Community: Millions of projects and libraries are available, easy to learn from.
- + Integrated Libraries: Supports easy communication via I2C, SPI, UART, PWM, and Analog interfaces.
- + Diverse Hardware: Many boards available for different needs (Uno, Mega, Nano, Due, etc.).
 - Disadvantages::
- + Limited Performance: Processing speed and RAM/Flash memory are restricted compared to ARM Cortex microcontrollers.
- + Lack of Security: No strong software/hardware security mechanisms; firmware can be easily copied.
- + Not Industry-Optimized: Difficult to handle applications requiring strict real-time constraints or industrial standards (CAN, Modbus, etc.).
- + Lack of Advanced Debugging: No strong hardware debugging tools like JTAG/SWD.

1.4. Strengths of the System Compared to Existing Products

1.4.1. Existing Smart Parking Projects

In recent years, many smart parking models have been implemented with the goal of automating vehicle entry and exit processes. Some common approaches include:

a. Pure RFID Systems

- Only use RFID tags to control vehicles.
- Advantages: Fast, low cost.
- Limitations: Cannot verify license plates; tags can be borrowed for fraudulent use.

b. Camera-Based License Plate Recognition Systems (ANPR)

- Cameras combined with AI/OCR to recognize license plates.
- Automatically record entry and exit times.
- Limitations: No user recognition; difficult to distinguish residents from visitors.

c. QR Code or Ticket-Based Systems

- Issue tickets to parkers or use QR codes.
- Limitations: Tickets can be lost, easily confused, slower processing speed than RFID.

d. Commercial Systems in Shopping Centers

- Combine cameras, sensors, and specialized barriers.
- Advantages: Very effective operation.
- Limitations: High cost; not suitable as a student project model.

1.4.2. Outstanding Strengths of the Project

Compared to previous projects, this project has significant improvements, fully meets practical requirements, and enhances safety:

a. Combination of Two Technologies: RFID + OCR Camera

- Both verify the validity of the tag and authenticate the actual license plate.
- Completely prevents “borrowed tag” fraud, increasing reliability.

b. User Classification (Residents vs. Visitors)

- Automatic system:
 - + Free/monthly card for residents.
 - + Minutely fee for visitors.
- This feature is absent in many other demo systems.

c. Automatic Fee Calculation by Vehicle Type

- Motorbike: 5,000 VND/minute
- Car: 10,000 VND/minute
- Clear algorithm; 100% automated fee calculation.

d. Smart Alerts

- Alerts for invalid tags.
- Alerts for mismatched license plates.
- Logs all violation cases.

=> This feature makes the system more practical and secure.

e. Complete Communication: Arduino → ESP32 → Server

- Arduino handles RFID + LCD + Servo.

- ESP32 manages WiFi and sends data to the website.
- The website processes logic and returns results to the security personnel.

f. Robust Management Website

- View real-time entry/exit cameras.
- Access entry/exit history.
- Manage RFID tags.
- Statistics by 14 days (Chart.js).
- Monitor the number of vehicles in the parking lot.

=> High completion level, similar to real-world systems.

g. High Practical Applicability

- Can be deployed for:
 - + Small apartment complexes
 - + Offices
 - + Small parking lots
- Low cost but highly efficient.

1.5. Factors Affecting the System

1.5.1. Distance and Orientation Between RFID Tag and Reader

- Each type of RFID reader has a limited signal reception range (typically 2–5 cm for RC522).
- If the tag is not presented in the correct orientation or is too far from the reader, reading errors may occur, disrupting the control process.

1.5.2. Electromagnetic Interference (EMI)

- Environments with many signal-emitting devices, high-power electrical equipment, or metal objects can cause interference, affecting the RFID tag's recognition ability.

1.5.3. Microcontroller Processing Speed

- If the program code is too slow or not optimized, the system may experience delays in gate response, signal activation, or incorrect information processing.

1.6. Main Components Used in the Circuit

1.6.1. ESP32 Module.

- Function: ESP32 is a microcontroller with integrated Wi-Fi and Bluetooth, widely used in IoT projects. With a dual-core processor at 230 MHz and approximately 520 KB of RAM, it allows fast and efficient processing. ESP32 supports multiple communication protocols such as GPIO, UART, I2C, SPI, PWM, ADC, DAC, etc., making it easy to interface with components like sensors, LCDs, servos, and relays.

- The device can connect to Wi-Fi or act as an access point, and supports Bluetooth Classic and BLE for wireless communication with phones or other devices. ESP32 also includes power-saving modes like deep sleep and light sleep, suitable for battery-powered systems. Additionally, it can function as a mini web server or send data to the Internet.

- ESP32 is commonly used in smart homes, smart parking systems, remote measurement, and device control. Users can program it easily using Arduino IDE, PlatformIO, or ESP-IDF. Thanks to its flexibility and rich feature set, ESP32 is an ideal choice for modern electronics and IoT projects.

a. Structure:

- Processor (CPU):
 - + Consists of 1 or 2 cores (dual-core) Xtensa LX6 or LX7.
 - + Speeds up to 230 MHz, supporting multitasking.
- Memory:
 - + Approximately 520 KB SRAM for temporary data.
 - + Flash memory from 3 MB to 16 MB for storing programs and long-term data (depending on the module).
- Wi-Fi and Bluetooth:
 - + Supports Wi-Fi 802.11 b/g/n (2.3 GHz).
 - + Supports Bluetooth Classic and BLE (Bluetooth Low Energy)
- Peripheral Interfaces:
 - + GPIO: Over 20 input/output pins for device control.
 - + UART, I2C, SPI: Communication with sensors, displays, external modules.

- + ADC (Analog to Digital Converter): Reads analog signals from sensors.
- + DAC (Digital to Analog Converter): Generates analog signals.
- + PWM: Controls LED brightness, servos, motors, etc.
- Power Management:
 - + Supports power-saving modes such as Deep Sleep and Light Sleep.
- Hardware Accelerators:
 - + Supports encryption/decryption such as AES, SHA, RSA, etc., for secure applications.
- Wi-Fi/Bluetooth Antenna:
 - + Built-in (onboard chip) or external antenna (via u. FL connector)

b. Functions:

- Integrated Wi-Fi and Bluetooth
 - + Enables easy wireless communication without additional modules.
 - + Supports both Bluetooth Classic and BLE (Bluetooth Low Energy).
- Powerful Processing
 - + Dual-core CPU with speeds up to 230 MHz
 - + Well-suited for applications requiring fast processing or multitasking.
- Multiple Communication Ports
 - + Supports standards such as GPIO, UART, I2C, SPI, ADC, DAC, PWM, etc
 - + Easily connects to sensors, relays, servos, LCD displays, and more
- Energy Efficiency
 - + Includes sleep modes (light and deep sleep) to extend battery life
 - + Suitable for applications requiring continuous operation with low power consumption.
- Low Cost – High Performance
 - + Much more powerful than Arduino Uno or ESP8266 with only a small price difference.
 - + Ideal for students, engineers, beginners, or professionals.
- Rich Software Support
 - + Programmable via Arduino IDE, PlatformIO, ESP-IDF, MicroPython, etc.
 - + Many free libraries available, with a large supportive community.

- Security Support
 - + Integrated hardware encryption (AES, RSA, SHA, etc.) for data protection.
 - + Suitable for IoT applications requiring high-level security.
- High Expandability
 - + Easily integrates with cloud platforms (Firebase, MQTT, HTTP), mobile apps, web servers, etc.
 - + Can function as a server or client device in a network
- Reason for Selection: Suitable for simple control projects, provides sufficient I/O pins for device connections, and is accessible for beginners.

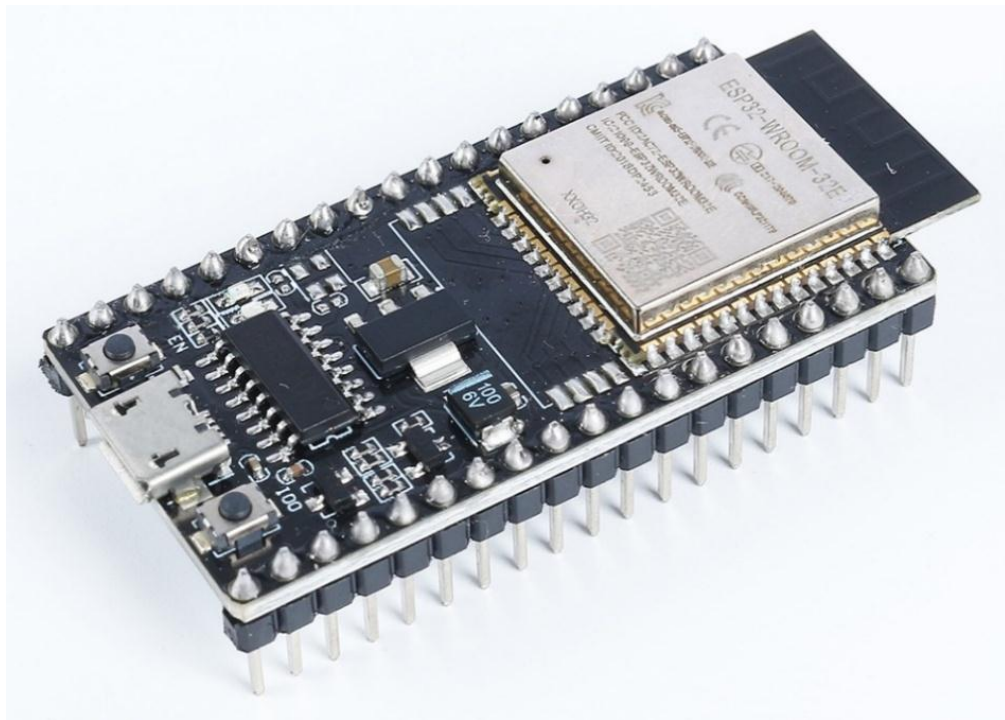


Figure 1.11: Actual Image of ESP32

Specifications

Table 1.2: ESP32 Technical Specifications

Parameter	Details
CPU	Dual-core Xtensa® LX6, maximum speed 230 MHz
RAM (SRAM)	~520 KB

Flash	3 MB to 16 MB (depending on module, typically 3 MB)
PSRAM	Available on some versions (ESP32-WROVER)
Wi-Fi	802.11 b/g/n (2.3 GHz), supports AP / Station / SoftAP
Bluetooth	Supports Bluetooth v3.2 BR/EDR and BLE (Bluetooth Low Energy)
GPIO	Up to 23 usable GPIO pins (depending on module and configuration)
ADC	18 channels, 12-bit resolution
DAC	2 channels, 8-bit resolution
PWM	Configurable on most GPIO pins
UART	2 UART interfaces
SPI	3 SPI interfaces (2 SPI, 2 HSPI/VSPI)
I2C	2 I2C interfaces
Wi-Fi Data Rate	Up to 150 Mbps
Operating Voltage	2.2V – 2.6V (typically 2.2V)

ESP32 Pinout Diagram

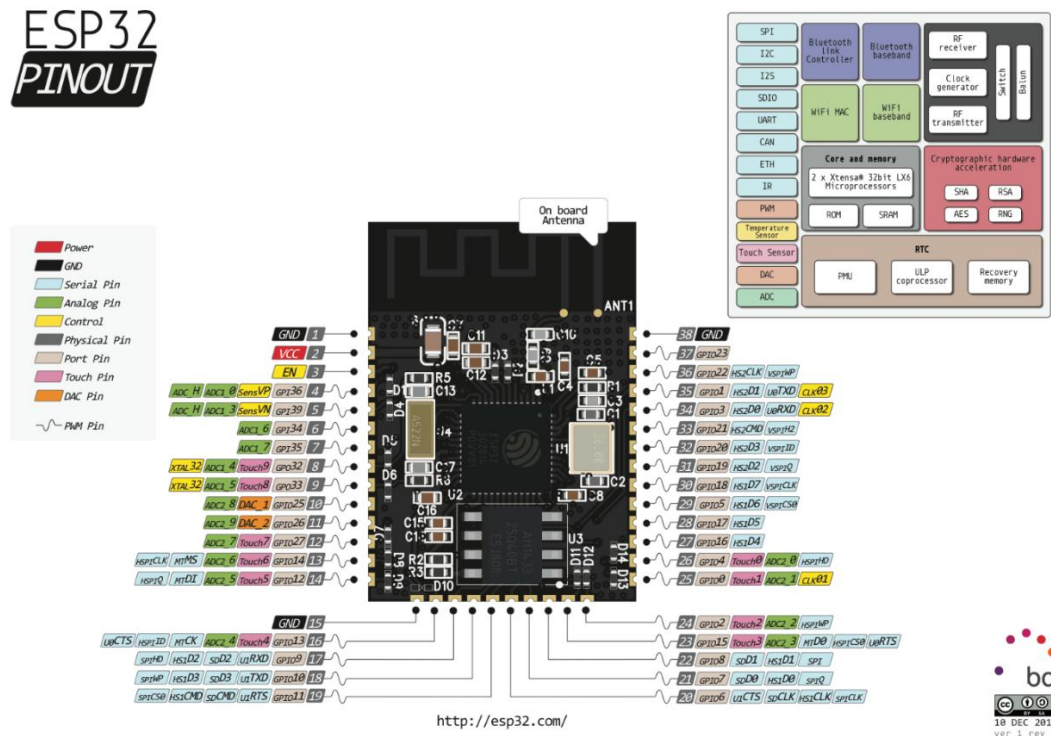


Figure 1.12: ESP32 Pinout Diagram

1.6.2. RFID RC522 Module

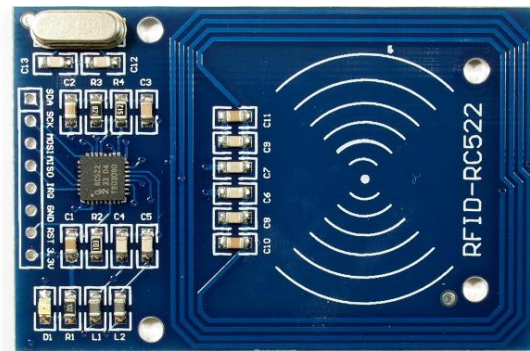


Figure 1.13: RFID RC522 Module

- Function: This is an RFID card reader operating at 13.56 MHz. It emits radio signals to activate passive RFID cards and reads data (ID) from the cards.
- Communication: Uses the SPI protocol to transmit data to Arduino.
- Reading Range: Approximately 2–5 cm.
- Advantages:
 - + Compact size, easy to integrate.

- + Low cost, widely available on the market.
- + Compatible with standard MIFARE 1K RFID cards..
- Reason for Selection: It is the most common RFID module for Arduino projects, stable operation, easy to program, and suitable for the simulation goals of this project.

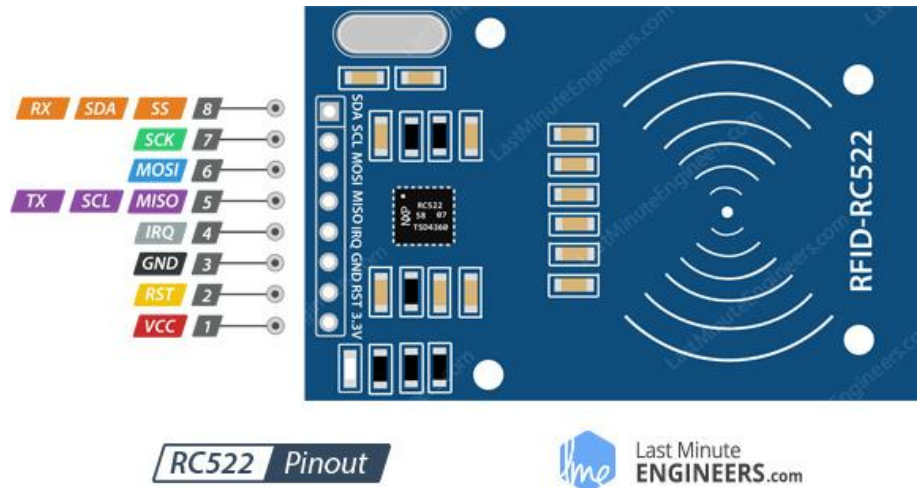


Figure 1.14: Pinout Diagram of the RFID RC522 Module

Remarks:

The RFID RC522 module is one of the most popular RFID readers today, especially for Arduino, ESP8266, STM32, and other microcontroller applications. With its compact size, low cost, and SPI communication support, it is highly suitable for automated identification systems such as parking management, access control, attendance tracking, etc.

- Advantages:
 - + High-speed SPI communication, easy to integrate with microcontrollers.
 - + Supports MIFARE 13.56 MHz RFID standard (common type).
 - + Stable reading range within 2–5 cm.
 - + Supported by Arduino IDE library (MFRC522).
 - + Low power consumption, stable operation.
- Disadvantages:
 - + Short reading distance, unsuitable for long-range scanning systems.
 - + Susceptible to interference in environments with metal or strong electromagnetic waves.

- + No electrostatic protection, easily damaged if handled carelessly during plugging/unplugging

Technical Specifications

Table 1.3: Technical Specifications of RFID RC522 Module

Parameter	Value
Main Chip	MFRC522 (NXP)
Operating Frequency	13.56 MHz
Communication Protocol	SPI (primary), I ² C, UART (configurable)
Operating Voltage	2.2V
Current Consumption	12–26 mA (during card reading)
Reading Distance	0–5 cm (optimal range: 2–5 cm)
Supported Card Types	ISO/IEC 14443 Type A (MIFARE S50, S70, Ultralight, Pro, DESFire)
Data Transfer Rate	Up to 323 kbit/s
Module Size	~30 mm x 60 mm
Built-in Antenna	Yes

Module RC522 Pinout

Table 1.4: Pin Connections of RFID RC522 Module

Pin	Name	Function
1	SDA / SS	Slave Select (SPI)
2	SCK	Clock (SPI)

Pin	Name	Function
3	MOSI	Master Out Slave In (SPI)
4	MISO	Master In Slave Out (SPI)
5	IRQ	Interrupt (rarely used, can be ignored)
6	GND	Ground (0V)
7	RST	Reset
8	3.3V	Power Supply (do not connect 5V directly)

Operating Principle:

The RC522 module operates based on the principle of near-field communication (NFC) to read information from passive RFID cards (without their own power supply).

1. Generating the Operating Magnetic Field:

- The RC522 generates an electromagnetic field at a frequency of 12.56 MHz through the integrated antenna on the board.
- When a passive RFID tag is within this field, it is powered by the magnetic field.

2. Activating the Card and Data Transmission:

- Once powered, the RFID tag responds by sending its unique identifier (UID) back to the RC522 reader.
- Data is transmitted via radio waves according to the ISO 13332A standard.

3. Communication with the Microcontroller:

- The RC522 receives the UID from the tag and forwards it to a microcontroller (e.g., Arduino) via the SPI (Serial Peripheral Interface) protocol.
- The microcontroller processes the received UID to verify access or perform subsequent control actions (such as opening a gate or displaying information).

1.6.3. RFID Card (MIFARE Tag).



Figure 1.15: RFID Card

- Function: Stores a unique ID code to be recognized by the RC522 RFID reader
- Characteristics:
 - + No power supply required (passive).
 - + Available in card form or keychain.
 - + Unique ID of 3 bytes (or 7 bytes), non-duplicated.
- Application in the project
 - + Represents vehicles entering and exiting.
 - + Multiple cards can be added to the list of valid cards in the Arduino program.

Technical Specifications of the MIFARE Classic RFID Card

Table 1.5: Technical Specifications of the RFID Card

Specification	Value
Name	MIFARE Classic 1K
Operating frequency	12.56 MHz
Memory capacity	1 KB EEPROM
Number of memory blocks	63 blocks (16 sectors, each sector 3 blocks)
Capacity per block	16 bytes
Card type	PVC card (credit card style, CR80) or keyfob

Specification	Value
Data transfer rate	106 kbit/s
Protocol	ISO/IEC 13332 Type A
Operating range	0 – 5 cm (depending on reader module and environment)
Write/erase cycles	> 100,000 cycles
Data retention	10 years or more
Chip type	NXP MF1S50 (or equivalent)
Security	2 keys A/B per sector, with 2 different access rights
Card dimensions	85.5 × 53 × 0.76 mm (ISO CR80 standard)

1.6.4. Servo Motor SG90

The SG90 Servo Motor is a popular mini servo, commonly used in small automation models. With a compact design and PWM control via microcontrollers like Arduino, the SG90 can rotate accurately from 0° to 180°. It has a built-in control circuit and position sensor, allowing it to maintain a stable angle according to the control signal.

In the parking lot simulation project, the SG90 is used to simulate the barrier arm: when a valid RFID card is scanned, the servo rotates to lift the barrier, then returns to its original position to close. With an operating voltage of 5V and simple usage, the SG90 is suitable for angle control in educational or prototype systems.



Figure 1.16: SG90 Servo Motor

Technical Specifications

Table 1.6: Technical Specifications of SG90 Servo Motor

Specification	Value
Operating voltage	3.8V – 6.0V
Rotation angle	0° – 180°
Control signal type	PWM (variable pulse width)
Current consumption	~100–250 mA
Weight	~9 g
Dimensions	22.2 × 11.8 × 21 mm
Rotation speed	~60°/0.1 s at 3.8V
Torque	~1.8 kg·cm at 5V

SG90 Servo Motor Pinout

Table 1.5: SG90 Servo Motor Pinout

Wire Color	Pin	Connect to	Function	Wire Color
Brown	GND (Ground)	GND (Arduino)	Ground connection	Brown
Red	VCC (Power)	5V (Arduino)	Provides power to the servo (3.8V–6V)	Red
Orange	Signal	Digital Pin D9 (e.g.)	Receives PWM pulses to control rotation angle	Orange

- Operating principle:

The SG90 servo motor operates based on position control using PWM (Pulse Width Modulation) signals. Each pulse repeats approximately every 20 ms, and the

pulse width determines the rotation angle of the servo: a 1 ms pulse corresponds to 0°, 1.5 ms to 90°, and 2 ms to 180°. Inside the servo, a control circuit combined with a potentiometer continuously compares the current position with the desired position from the control signal. If there is a discrepancy, the motor rotates through a geared reduction system to bring the shaft to the correct angle and then holds it in place. This feedback mechanism allows the servo to adjust precisely and maintain a stable position as required

1.6.5. Buzzer (5V chip buzzer).

- The buzzer is a simple sound-emitting device, commonly used for alerts, confirming actions, or signaling errors in automated control systems. In this project, the buzzer notifies the status when a user scans an RFID card, for example:

- Short beep: valid card → gate opens.
- Long or multiple beeps: invalid card → access denied



Figure 1.17: Buzzer module

Buzzer classification

Table 1.6 Buzzer classification

Type	Characteristics	Control method
Active	Built-in oscillator, only requires power to sound	<code>digitalWrite(pin, HIGH)</code>
Passive	Requires control with audio pulses (PWM)	Use <code>tone(pin, freq)</code> in Arduino

Buzzer pinout

Table 1.7: Buzzer pinout

Pin	Function
QUA	Digital signal output
VCC	Power supply
GND	Ground

Operating principle of the buzzer:

- The buzzer works by converting electrical energy into mechanical oscillation to produce sound. When powered, the current passes through a coil to generate a magnetic field, which vibrates a metal diaphragm or ceramic disc inside the buzzer, producing sound at a specific frequency.

- There are two main types of buzzers:

- + Active buzzer: has a built-in oscillation circuit, so it only needs power (HIGH level) to produce sound automatically.
- + Passive buzzer: does not generate sound on its own and requires a control signal (PWM) from the Arduino to create oscillation, typically using the `tone ()` function with the desired frequency.

Depending on the buzzer type, the Arduino provides either a control signal or simple current to make the buzzer emit sound signals for notification purposes.

1.6.6. Display Block.

1.6.6.1. 16x2 LCD display.

- Overview: Nowadays, LCD (Liquid Crystal Display) devices are widely used in microcontroller applications. LCDs offer several advantages over other types of displays. They can display a wide variety of characters and graphics (letters, numbers, and symbols) and can be easily integrated into circuits using different communication protocols. LCDs consume very few system resources and are low-cost.

- Advantages:

- + Can display diverse and clear characters and graphics (letters, numbers, symbols)
- + Easy to integrate into circuits via various communication protocols

- + Low system resource consumption and inexpensive
- Disadvantages:
 - + Due to low pixel density, outdoor sunlight visibility is poor, and moving pixels may be noticeable.



Figure 1.18: 16x2 LCD Display

Technical Specifications

Table 1.8: 16x2 LCD Specifications

Parameter	Value/Characteristics
Display size	2 lines, 16 characters per line (total 32 characters)
Dot matrix	5x8 dot matrix per character
Operating voltage (VDD)	3.5V ~ 5.5V (typically 5V)
Display voltage (VO)	~0–0.7 VDD (adjustable via potentiometer)
Current consumption	~1–2 mA (excluding backlight)
LED backlight	~15–20 mA (depending on type)
Interface	3-bit or 8-bit parallel

Parameter	Value/Characteristics
Main driver IC	HD33780 or compatible
Supported characters	ASCII, user-defined characters (CGRAM)
Operating temperature	-20°C ~ +70°C
Viewing angle	~±30 degrees
Module dimensions	~80 mm x 26 mm x 10 mm (depending on manufacturer)
Display area	~66 mm x 16 mm

1.6.6.2. I2C Module

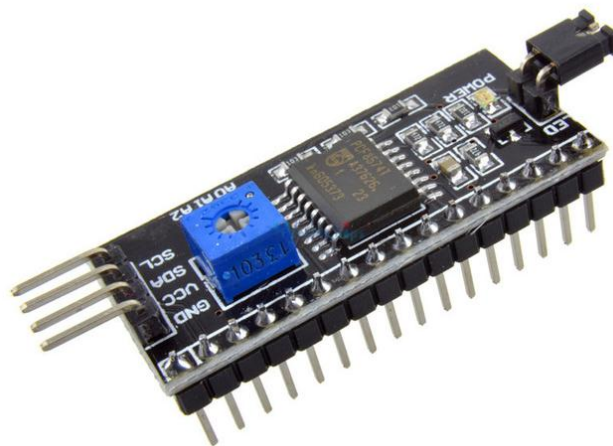


Figure 1.19: I2C LCD 16x2 Module

The I2C adapter module for LCD simplifies the connection between the LCD and microcontroller, reducing the number of pins used from 6 to just 2 (SCL and SDA). This saves microcontroller pins and makes the circuit more compact. The I2C module supports LCDs using the HD33780 driver (such as LCD1602, LCD2003, etc.) and can communicate via the I2C interface, compatible with most modern microcontrollers.

1. Specifications:
2. Operating voltage: 2.5–6V DC
3. Supported screens: LCD1602, 1603, 2003 (HD33780 driver)

4. Communication: I2C
5. Default address: 0x27 (adjustable via A0/A1/A2 solder jumpers)
6. Dimensions: 31.5 mm (D) × 19 mm (W) × 15.2 mm (H)
7. Weight: 5 g
8. Integrated jumper to control LCD backlight or disable it
9. Built-in potentiometer to adjust LCD contrast

1.6.7. 5V Relay Module

A relay is an electromagnetic switch that operates using a small current to control a much larger current. At the heart of a relay is a coil (electromagnet) – when current flows through the coil, it generates a magnetic field, turning it into a temporary magnet. This magnetic force actuates the contacts, allowing the relay to open or close circuits with higher currents.

The 5V relay module is an electronic switching device used to control high-voltage loads (AC or DC) through low-voltage control signals from microcontrollers such as Arduino, ESP32, or Raspberry Pi. The relay acts as an ON/OFF switch controlled by LOW or HIGH signals on the control pin.



Figure 1.20: 5V Relay Module

- The relay consists of three main sections, each with a specific function:
 - + Input section (receiver mechanism): Receives input signals and converts them into appropriate signals for the intermediate section.
 - + Intermediate section (control mechanism): Takes signals from the input section and converts them into control signals for the relay.
 - + Output section (actuator mechanism): Acts on the circuit, allowing the relay

to open or close the electrical contacts.

- Specifications:
 - + Operating voltage: 5V DC
 - + Max operating current: 80 mA
 - + Max AC load voltage: 250V
 - + Max DC load voltage: 30V
 - + Max load current: 10A
 - + Max power: 2500W
 - + Operating temperature: -20°C – 125°C
 - + Signal selection: Trigger LOW or HIGH via jumper.
- Advantages:
 - + Can control high-current loads.
 - + Good isolation from microcontroller → safer operation.
 - + Easy to use, controlled with digital HIGH/LOW signals.
 - + Durable and low-cost.
- Disadvantages:
 - + Mechanical relay, produces clicking sound when switching.
 - + Switching speed slower than MOSFETs.
 - + May generate electromagnetic interference when controlling large loads.
 - + Not suitable for PWM control (e.g., servos).
=> Relay is only suitable for ON/OFF control, not for controlling angles.

1.6.8. LM2596 Step-Down (Buck) Module

The LM2596 module is a DC-DC Buck Converter using the LM2596 IC from Texas Instruments. It can step down an input voltage from 4.5V–40V to a lower, stable voltage with high efficiency and low heat generation. This module is widely used in electronics projects, especially when converting power from batteries, adapters, or 12V sources down to 5V for microcontrollers or sensors.

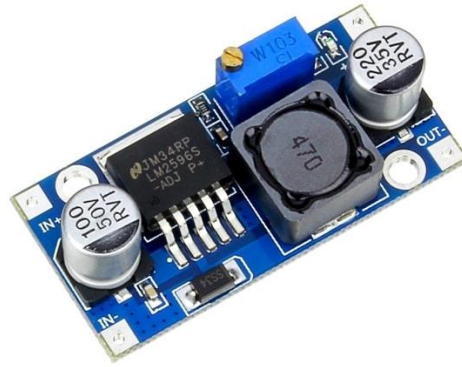


Figure 1.21: LM2596 Step-Down Module

1. Function:

- Converts high DC voltage → lower DC voltage as required.
- Provides stable power for sensitive devices such as:
 - + Arduino, ESP32
 - + RFID RC522
 - + Servo motors, relays
 - + Cameras or other peripheral modules
- Protects the system from overvoltage that could damage microcontrollers

2. Technical Specifications

Table 1.9: Specifications of LM2596 Step-Down Module

Parameter	Value
Input voltage	4.5V – 40V DC
Output voltage	1.25V – 37V DC (adjustable via potentiometer)
Maximum current	2A – 3A (depending on version)
Efficiency	Up to 92–96%
Switching frequency	150 kHz
Protection	Reverse polarity diode, voltage stabilization

3. Structure

- The LM2596 module consists of:

- + LM2596 IC (Buck switching regulator)
- + 150 μ H inductor
- + Schottky diode for reverse polarity protection
- + Input and output filter capacitors
- + Potentiometer for voltage adjustment
- Connection pins:
 - + VIN+ / VIN- $\rightarrow$ input voltage
 - + VOUT+ / VOUT- $\rightarrow$ stepped-down output voltage

4. Advantages

- High efficiency $\rightarrow$ energy-saving, low heat generation.
- High current capacity (up to 2–3A) $\rightarrow$ sufficient for multiple devices.
- Low cost, easily available, easy to use.
- Smooth and precise voltage adjustment.
- Stable operation even with changing load.

5. Disadvantages

- Cannot boost voltage $\rightarrow$ only steps down.
- Non-isolated output (unlike isolated switching power supplies).
- Continuous load above 2A may require heat dissipation.

1.6.9. Raspberry pi 3

Raspberry Pi 3 Model B is a mini embedded computer (single-board computer) developed by the Raspberry Pi Foundation. Despite its compact size, it integrates full computer functionalities, including CPU, RAM, Wi-Fi, Bluetooth, USB ports, HDMI, and its own operating system (Raspberry Pi OS). Raspberry Pi 3 is commonly used in IoT projects, real-time control, image processing with cameras, and building smart systems.



Figure 1.22: Raspberry Pi 3

1. Functions

- Process data and run an operating system (Linux/Raspberry Pi OS).
- Receive data from cameras and perform image processing (OCR, license plate recognition, AI).
- Communicate with devices such as:
 - + Arduino/ESP32 (via UART, USB, TCP/IP)
 - + PiCam / USB cameras
 - + Sensor modules, relays, LEDs...
- Connect to the Internet to send data to a server (HTTP, MQTT, WebSocket).

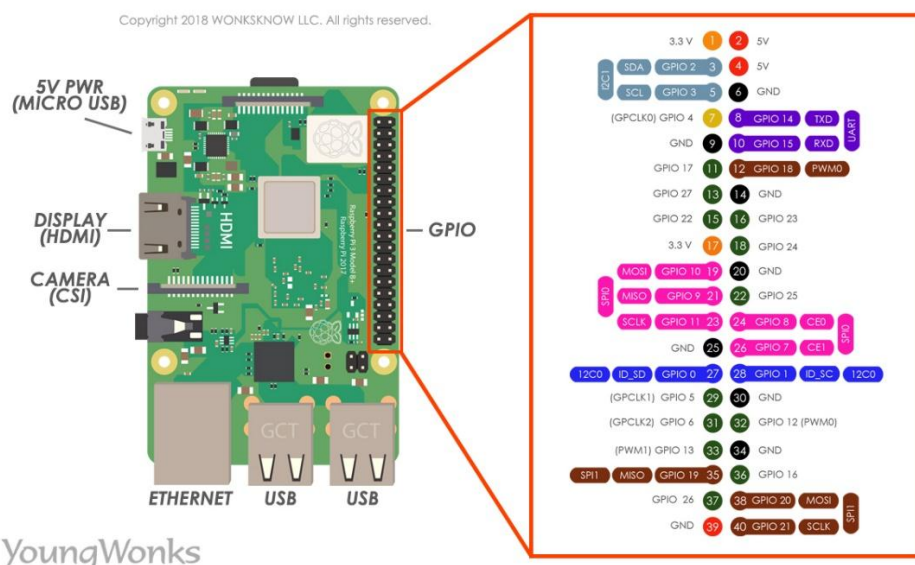


Figure 1.23: Pinout diagram of Raspberry Pi 3

Table 1.10: Specifications

Component	Specification
CPU	Broadcom BCM2837 – Quad-core ARM Cortex-A53 1.2 GHz
RAM	1GB LPDDR2
WiFi	802.11 b/g/n (2.4GHz)
Bluetooth	Bluetooth 4.1 + BLE
USB Ports	4 × USB 2.0
HDMI	1 × full-size HDMI
GPIO	40 general-purpose GPIO pins
Power Supply	5V–2.5A via Micro USB
Camera Interface	CSI port (for Pi Camera)
Dimensions	85 × 56 × 17 mm

2. Advantages

- Compact size, easy to integrate into models.
- Higher performance compared to microcontrollers like Arduino/ESP32.
- Supports image processing, video, AI, and machine learning.
- Built-in WiFi and Bluetooth.
- Runs a full operating system, making Python programming easy.
- Large and active support community.

3. Disadvantages

- Higher power consumption than Arduino/ESP32 (requires stable 5V–2.5A supply).
- Slow boot time due to OS startup.

- SD card can be damaged if power is suddenly cut off.
- Not suitable for real-time tasks (e.g., direct servo control) => suitable for image processing, not direct motor control

1.5.10. ICON Webcam Q-20

ICON Q-20 Webcam is a digital video capture device commonly used with computers and small surveillance systems. It is an affordable webcam with good image quality, suitable for vehicle license plate capture in parking lot models.



Figure 1.24: ICON Webcam Q-20

Key Features

- Resolution 2K or 4K: Provides sharp images, suitable for reading license plates or monitoring vehicles.
- Wide-angle lens: Covers a large area, ideal for placement at vehicle entry/exit points.
- Auto-focus: Maintains clear images even when vehicles are moving.
- Built-in microphone: Can capture audio if needed.
- Plug & Play: Connects via USB 2.0/3.0, no driver installation required on Raspberry Pi or Windows.
- Cross-platform compatibility:
 - + Windows / macOS
 - + Linux
 - + Raspberry Pi OS

=> Works well with OpenCV for license plate recognition.
- Typical Specifications

- + Resolution: 2K or 4K (depending on version)
- + Field of view: ~90°
- + Connection: USB 2.0
- + Frame rate: 25–30 fps
- + Focus: Automatic
- + Low-light support: Yes (Auto Exposure)

1.7 Conclusion of the Chapter

Chapter 1 has presented the theoretical foundation and technical platforms for building a smart parking system. The content of the chapter includes an overview of IoT systems, the basic architecture of an IoT system, software used in the system, as well as an analysis of the advantages of this system compared to traditional solutions.

Additionally, the chapter discussed factors affecting the system's operational efficiency, such as the distance and orientation of devices, electromagnetic interference (EMI), and the processing speed of microcontrollers. These factors play a crucial role in ensuring stability, accuracy, and reliability in real-world environments.

Finally, Chapter 1 introduced in detail the main components used in the project, including the ESP32, RFID RC522 module, MIFARE RFID cards, servo motor, buzzer, display unit, relay, and power modules. Analyzing the function and operating principles of each component provides a solid foundation for designing, developing algorithms, and implementing the system in the subsequent chapters.

CHAPTER 2: SYSTEM ANALYSIS AND DESIGN

2.1. Functions and Selection of Main Components for Each Block

2.1.1. Identification Block.

- a. Function: Identify vehicles via RFID cards with unique identification codes.
- b. Selected Components:
 - RC522 RFID Module (12.56 MHz)
 - + Low cost, easy to integrate with Arduino.
 - + Reading distance ~2–5 cm, sufficient for simulation models.
 - + SPI communication ensures fast and stable data transfer.

2.1.2. Display Block

- a. Function: Notify system operating status to users (valid/invalid card, welcome message, etc.).
- b. Selected Components:
 - LCD 1602 (5V, green)
 - + Displays 2 lines × 16 characters, suitable for concise information.
 - + Easy to interface with Arduino (3-bit communication or I2C if using an expansion module).

2.1.3. Central Processing Block.

- a. Function: Process data from the RFID reader, compare codes, make decisions, and control output devices.
- b. Selected Components:
 - Esp32
 - + Easy to program, large support community.
 - + Has sufficient pins to connect devices: RC522, servo, LCD, buzzer.
 - + Suitable for small systems, simulations, and educational purposes.

2.1.4. Gate Control Block .

- a. Function: Open or close the parking gate according to the decision from the processor.
- b. Selected Components:

- Servo Motor SG90 (180°)
 - + Compact, easy to control via PWM.
 - + Suitable for simulating gate movement in small models.
 - + Rotates accurately according to programmed angles.

2.1.5. Warning Block.

a. Function: Produce sound or light signals to notify system status (incorrect card detection, gate opening, errors, etc.).

b. Selected Components:

- Buzzer 5v
 - + Small size, operates stably at 5V.
 - + Emits simple warning signals.

2.1.6. Power Block

a. Function: Provide stable power to Arduino, ESP32 (if any), servo, relay, and Raspberry Pi 3; ensure no voltage drop when servos operate or when the camera + Pi consume power.

b. Selected Components:

- 5V DC power supply (e.g., 5V 3A or 5V 4A adapter) for Raspberry Pi and other 5V devices.
- 5V step-down module (buck converter) (e.g., LM2596 / MP1584) to convert 9–12V input down to stable 5V for servo/relay/ESP32 (if using a common 12V source).

2.1.7. Relay Block

a. Function: Switch power on/off for devices (e.g., control power for an actual gate motor, or 5V circuit switch for peripheral devices).

b. Selected Components:

- 1-channel 5V relay module

2.1.8. Camera & Image Processing Block

a. Function: Record/compare license plates, monitor the gate, authenticate users/vehicles (advanced).

b. Selected Components:

- Raspberry Pi 3 (RPi 3 B/B+) as the image processing unit.
- 2 camera

Applications: Store images when scanning cards, license plate recognition (OCR), authenticate card/image matches, save image logs, monitor.

Connection to ESP32 / main system: Raspberry Pi communicates with ESP via UART (TX/RX), USB serial, or network (MQTT/HTTP). Pi processes images and sends results (OK/NOT OK) to ESP for gate operation.

2.2. Hardware Design for the System.

2.2.1. Design Objectives

- a. Automatic Vehicle Identification:
 - Design an RFID card reading system mounted on vehicles, ensuring accurate and fast recognition when vehicles enter/exit the parking lot.
- b. Gate Mechanism Control:
 - Use a servo or motor to control the barrier/gate automatically based on data from the RFID card.
- c. Visual Information Display
 - Configure a 16x2 LCD screen to display vehicle type, card status, parking duration, and fee (if any).
 - Integrate a buzzer to alert for invalid cards or operational errors.
- d. Communication and Data Transmission:
 - Arduino receives signals from the RFID reader and transmits data to the ESP32 via UART.
 - ESP32 handles Wi-Fi connection to send data to the management system on the website.
- e. Ensure Stability and Expandability:
 - Design the electrical circuit and hardware layout appropriately for easy maintenance, upgrades, and expansion to multiple entry/exit gates or multiple types of vehicles.

2.2.2. Overall System Block Diagram

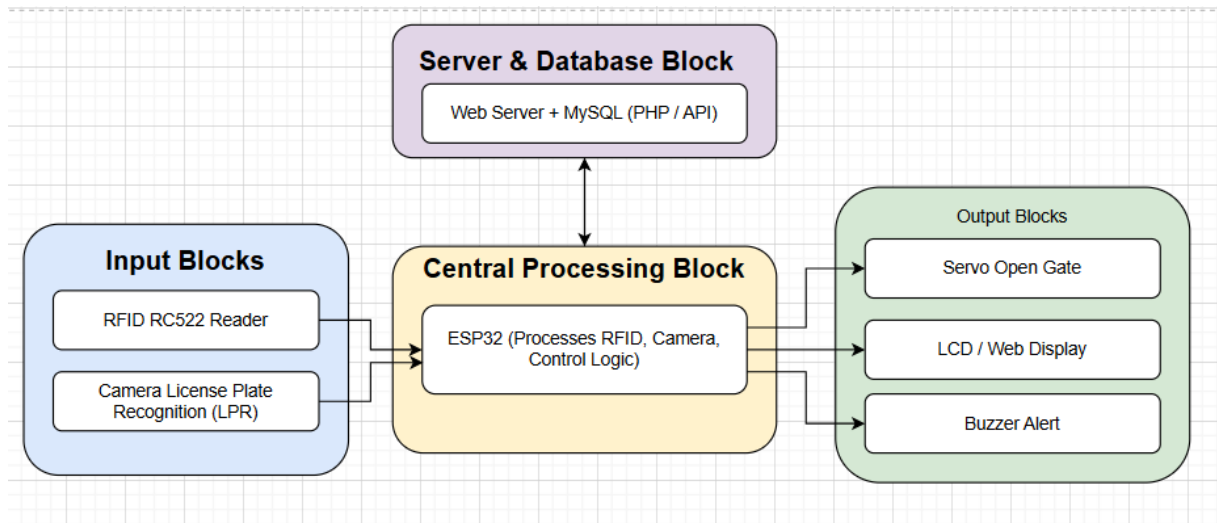


Figure 2.1: Overall Block Diagram

2.2.3. System Block Diagram

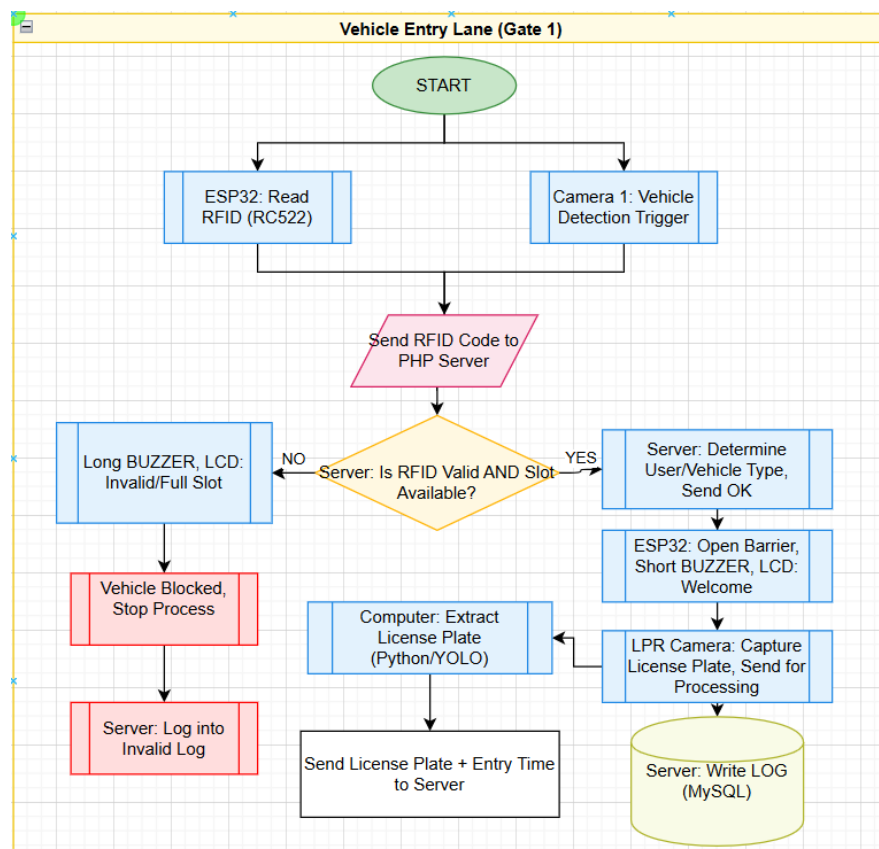


Figure 2.2: System diagram when the vehicle enters

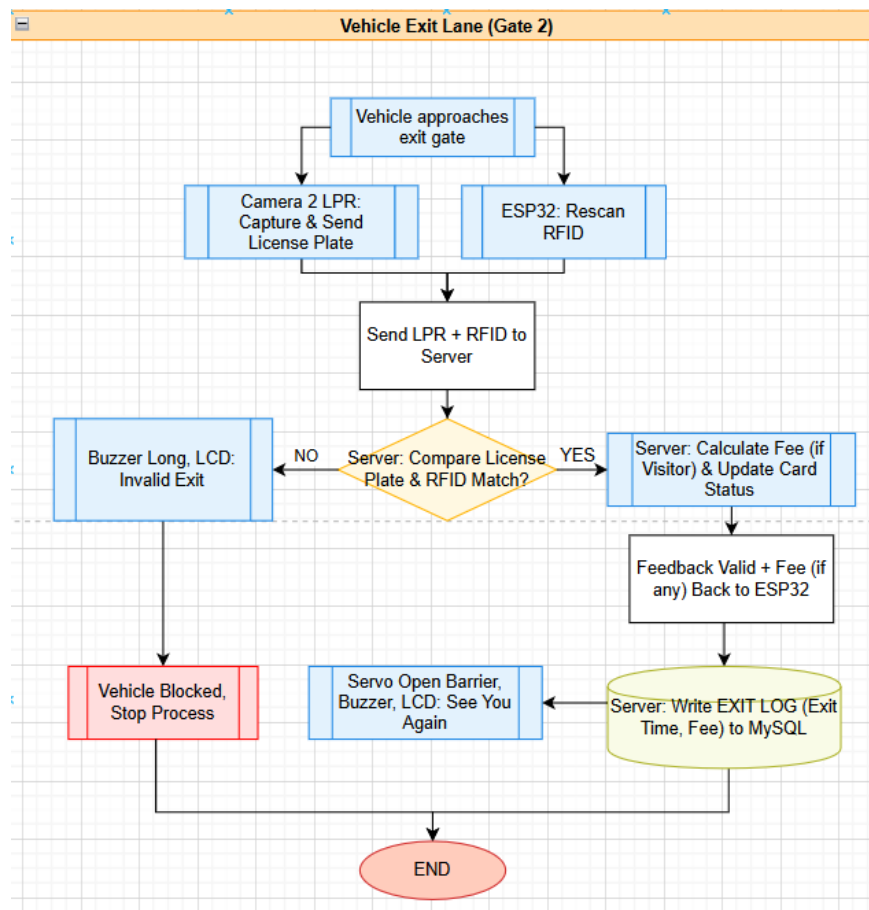


Figure 2.3: System diagram when the vehicle exits

2.2.3. Functions and Operating Principles of Each Block

a. RFID card

- Function:

The RFID card attached to the vehicle contains unique identification information (UID). The system uses this card to distinguish between valid and invalid vehicles entering the parking lot.

- Operating Principle:

When the vehicle approaches the gate, the card is activated by the electromagnetic field emitted from the RFID reader. The card responds with its UID to the reader, sending this data to the Arduino or ESP32 for further authentication. The RFID card does not require a separate power source and operates using energy induced from the reader.

b. RFID Reader (RC522)

- Function:

Receive the UID data from the RFID card. This is a crucial intermediary device

that enables the system to “identify” each vehicle.

- Operating Principle:

The reader emits an electromagnetic field to activate the card. When the card responds with its UID, the reader captures the signal and decodes it, then transmits the UID data via SPI or UART to the Arduino/ESP32. The reader operates using a wireless read/write mechanism at a frequency of 13.56 MHz.

c. Camera 1 (Vehicle Entry Monitoring)

- Function:

Monitor vehicles entering the parking lot, capture images or videos of the vehicle, and optionally integrate license plate recognition to store information.

- Operating Principle:

When the Arduino detects a vehicle approaching the gate (via RFID signal or physical sensor), Camera 1 is automatically activated to capture images. These images are sent to the Arduino or a processing computer to be stored, cross-referenced with the card UID, and updated in the parking management system.

d. Camera 2 (Vehicle Exit Monitoring)

- Function:

Monitor vehicles exiting the parking lot, capture vehicle images, and record exit time.

- Operating Principle:

When a vehicle approaches the exit gate, Camera 2 automatically captures images and sends the data to the Arduino/ESP32. The images, combined with the UID from the card, allow the system to verify whether the exiting vehicle is valid, cross-check with entry history to calculate fees, or trigger alerts for discrepancies.

e. ESP32

- Function:

Central processing unit for data from the RFID reader, cameras, and sensors; controls peripheral devices such as the barrier, buzzer, and LCD; sends processed data to the server.

- Operating Principle:

- + Receives UID from RC522 and data from sensors.

- + Checks the card data against the valid list (stored locally or on the server).
- + Controls barrier opening/closing, buzzer alerts, and LCD display for vehicle/card/fee information.
- + Sends processed data (UID, status, time, images) to the server via UART.

f. Servo Motor / Barie

- Function: Open and close the entrance/exit gate of the parking lot.
- Operating Principle:

The servo receives control signals from the Arduino. If the card is valid and the vehicle is permitted to enter, the servo rotates to lift the barrier. If the card is invalid or the parking lot is full, the servo does not receive the signal, and the barrier remains in place, preventing vehicle entry.

g. Buzzer & LCD

- Function: Provide audio alerts and display visual information.
- Operating Principle:

Controlled by the ESP:

- + Buzzer: Emits long or short sounds to alert for invalid cards, full parking lot, or system errors.
- + LCD: Displays vehicle information, card type, parking lot status, parking fee, and warnings for invalid vehicles

h. ESP32 (UART + Wi-Fi)

- Function: Receive data from Arduino and transmit it to the server or website.
- Operating Principle:

ESP32 connects to the Arduino via UART to receive UID, card status, and images from cameras. Then, ESP32 connects to Wi-Fi and sends HTTP requests to the PHP/MySQL server to store data and synchronize information with the parking management website.

i. Server / Website / MySQL

- Function: Store vehicle and card data, entry/exit history, and support statistics and parking management.
- Operating Principle:

- + Receive data from ESP32 via HTTP requests.
 - + Verify card validity based on the valid_rfids database.
 - + Store data in MySQL tables: logs (entry/exit history), invalid_logs (invalid cards), and update current vehicle count in the parking lot.
 - + Provide a website interface for management to monitor: vehicle list, entry/exit times, daily/weekly/monthly statistics, and alerts for full parking or invalid cards.
- The system operates based on the RFID card recognition process and data processing through the Arduino microcontroller to control peripheral devices such as the LCD screen, servo motor, and buzzer. Specifically:
- The user brings the RFID card close to the RC522 reader. The reader uses radio waves (frequency 12.56 MHz) to communicate with the RFID card. When the card is within sufficient range, the RC522 reads the unique ID on the card.
 - The RC522 module transmits the card ID to the Arduino Uno microcontroller via SPI communication. The Arduino processes this ID and compares it with a pre-programmed list of valid IDs.
 - If the card ID is valid:
 - + The Arduino controls the SG90 servo motor to rotate an angle (usually 90–180 degrees) to open the simulated gate.
 - + Simultaneously, the 16x2 LCD displays a welcome message or "Valid Card – Gate Open."
 - + The buzzer or LED may be activated briefly as a success confirmation signal.
 - If the card ID is invalid:
 - + The gate does not open, and the servo does not rotate.
 - + The LCD displays "Invalid Card" or a similar message.
 - + The buzzer emits a short warning beep.
 - + After a few seconds, the servo motor returns to its original position (gate closed), and the system is ready to receive the next card.

2.1.4. Circuit Schematic Design

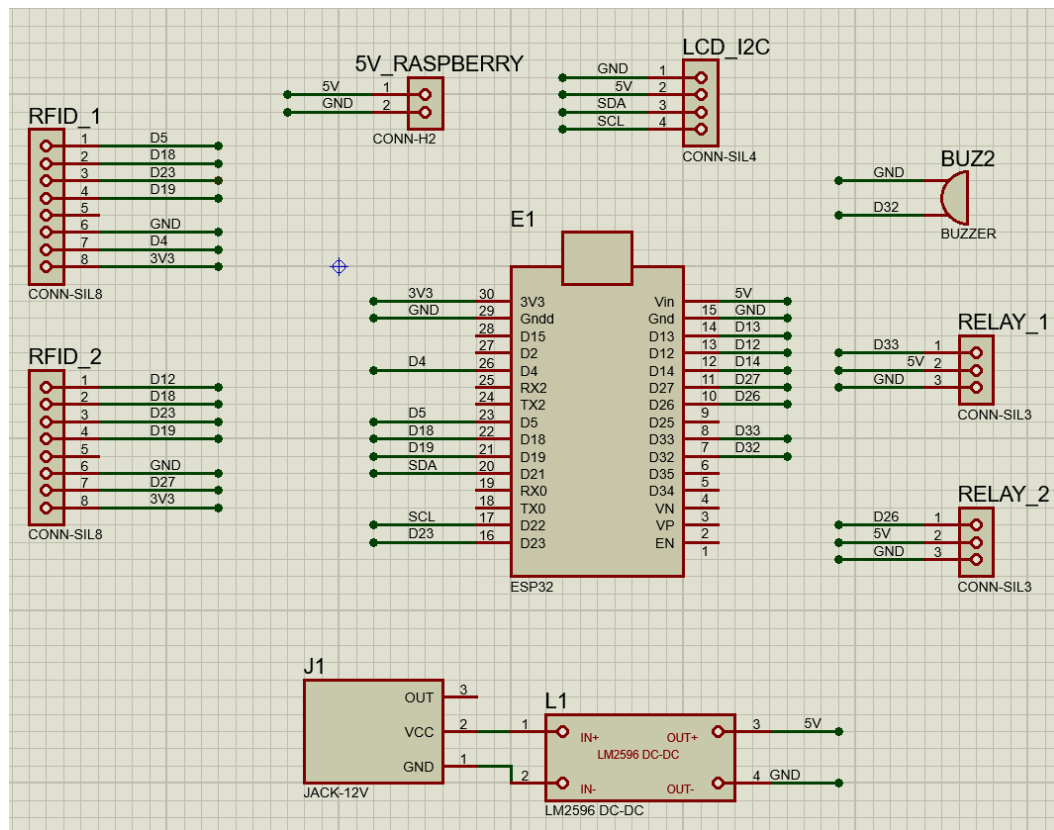


Figure 2.4: System Schematic Diagram

This is the control circuit for the automatic vehicle recognition system using ESP32, operating based on the RFID card recognition principle. When a vehicle approaches the gate, the driver brings the RFID card close to the RC522 reader. The RC522 RFID module emits an electromagnetic field to activate the card and receives the card's unique UID, then transmits this data to the Arduino via SPI communication.

The ESP32 reads the UID and compares it with a pre-registered list of UIDs in memory to determine whether the vehicle is permitted to enter. If the card is valid, the Arduino displays "Access Granted" or "Welcome" on the 16x2 I2C LCD, while simultaneously activating the buzzer to emit a short alert sound to notify the driver. Conversely, if the card is invalid, the Arduino displays "Access Denied" and may trigger a longer warning sound.

The circuit is compactly designed, using an I2C LCD that requires only two data lines (SDA and SCL) to minimize wiring, while the buzzer is directly controlled via a digital pin to emit sound when needed. This automated solution enables fast and accurate vehicle recognition and control for parking lots or restricted areas without manual intervention. It can be expanded with an automatic barrier or integrated with entry/exit history logging to enhance management..

2.1.5. PCB Circuit

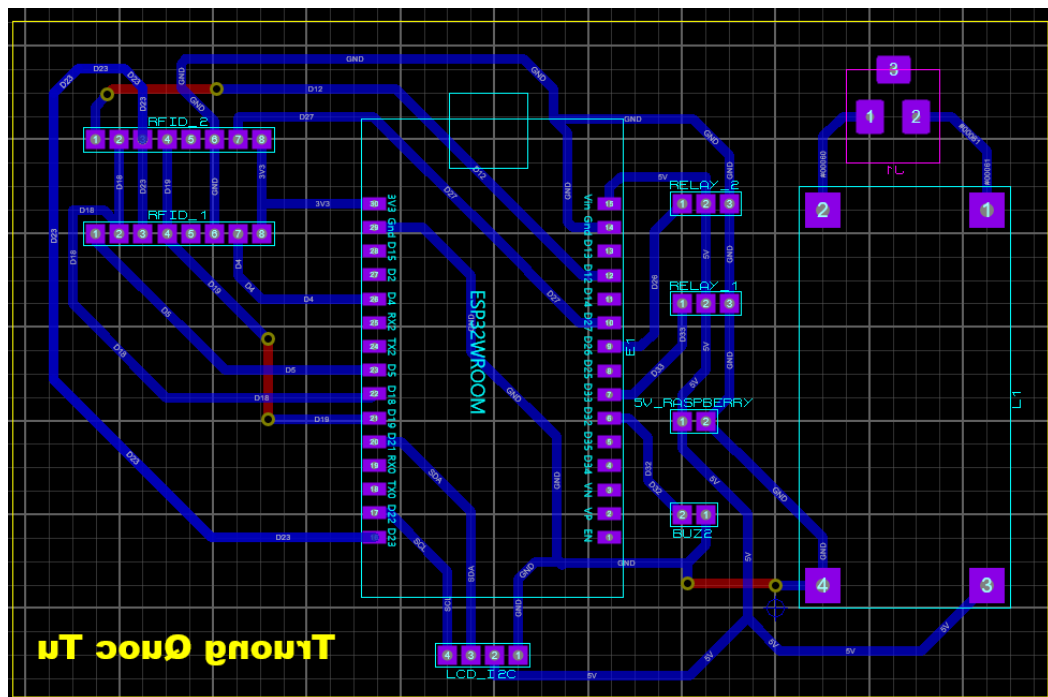


Figure 2.5: Wiring Diagram of the Circuit

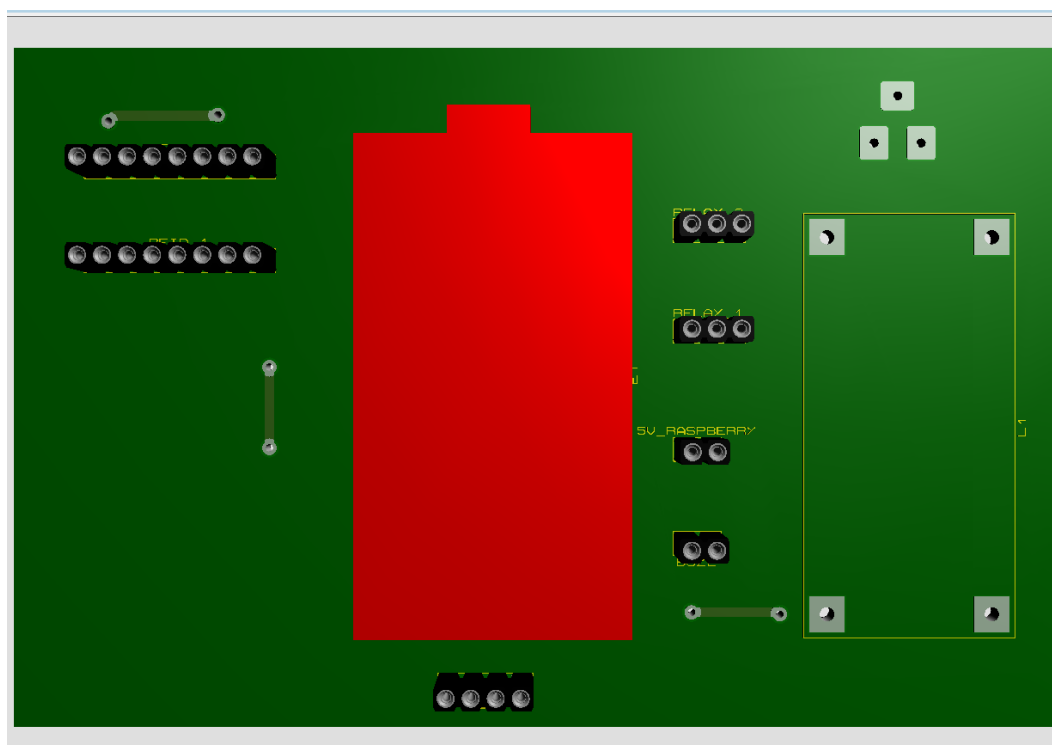


Figure 2.6: 2D Image of the Circuit

2.2. Analyze and design software for the system

2.2.1. Program Algorithm Flowchart

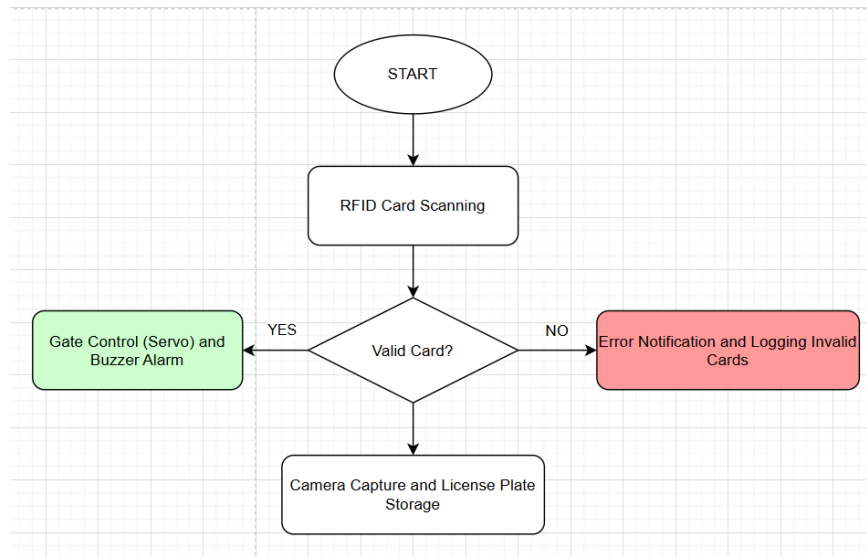


Figure 2.7: Algorithm Flowchart for Vehicle Entry

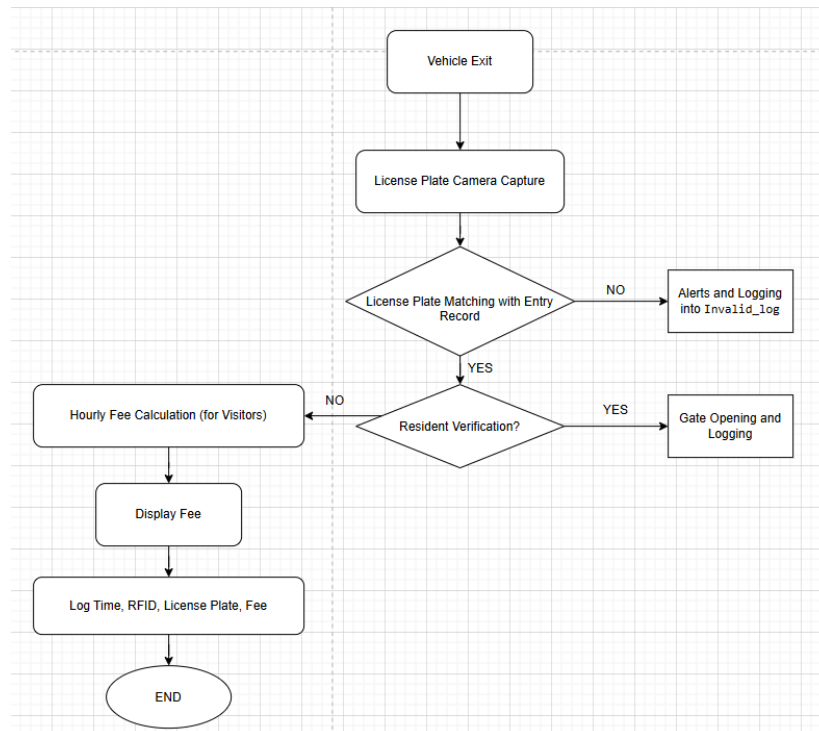


Figure 2.8: Algorithm Flowchart for Vehicle Exit

2.2.2. Program Algorithm Explanation::

1. System Initialization

When the system is powered on, the components are initialized sequentially:

- a ESP32 performs:

- Connects to the local Wi-Fi network.
- Initializes the RC522 RFID reader to be ready to read card UIDs.
- Initializes the I2C LCD to display system status.
- Initializes the buzzer and servo to provide alerts and control the gate.
- Synchronizes real-time (NTP) to ensure accurate entry/exit timestamps.

b Raspberry Pi 3:

- Boots the Raspberry Pi OS..
- Initializes 2 cameras:
 - + Camera 1: monitors the entry lane.
 - + Camera 2: monitors the exit lane.
- Starts the image processing service (Python + OpenCV + OCR/AI).

2. RFID Card Reading

When the user brings the RFID card into the scanning area

- ESP32 reads the card UID from the RC522.
- Retrieves the current timestamp from the NTP system.
- Sends data to the Web Server (PHP) via HTTP GET/POST with parameters:
 - + uid
 - + timestamp
 - + status = CHECK

3. Validity Check

The Web Server processes the request from ESP32:

Checks the UID in the valid_rfids table.

- If valid:
 - + Responds with "VALID" along with:
 - user_type
 - vehicle_type
- If invalid:
 - + Responds with "INVALID"

- Upon receiving "INVALID," ESP32:
 - + Displays “Invalid Card” warning on the LCD.
 - + Sounds an alert via buzzer.
 - + Does not activate the servo to open the gate.
 - + Logs the event in invalid_logs (including timestamp and camera image if available).

4. Determine Action (Entry / Exit)

If the card is valid:

- The server queries the logs table to get the latest status of the UID:
 - + If no record exists or the last record is status = 'out'
=> System determines the action as Entry.
 - + If the last record is status = 'in'
=> System determines the action as Exit.
- The action information is sent back to ESP32.

5. License Plate Recognition via Camera

Based on the determined action:

- If Entry:
 - + Raspberry Pi activates Camera 1 (entry lane).
 - + Captures the vehicle license plate image.
 - + Processes the image using OpenCV/OCR for plate recognition.
 - + Sends the license plate result and image path to the Web Server..
- If Exit:
 - + Raspberry Pi activates Camera 2 (exit lane).
 - + Captures the vehicle license plate image.
 - + Recognizes and compares the plate with the entry record.
 - + If plates match → allows exit.
 - + If plates do not match → triggers an alert..

6. Send and Store Data on Server

ESP32 and Raspberry Pi send data to the server:

- Information is stored in the logs table, including:
 - + RFID card UID
 - + Status (in / out)
 - + Entry/exit timestamp
 - + Vehicle license plate
 - + Entry/exit images
 - + User type and vehicle type
 - + Parking fee (if visitor)

7. Hardware Control

When all conditions are met:

- ESP32:
 - + Displays user information and status (Entry / Exit) on the LCD.
 - + Emits a confirmation beep via buzzer.
 - + Controls the servo to open the gate.
 - + After a preset time, the servo automatically closes.
- If errors occur (invalid card, plate mismatch):
 - + Servo does not open.
 - + Alerts are triggered and events are logged.

8. Waiting State

After processing a vehicle:

- LCD displays the default message.
- System returns to waiting state, ready to process the next card.

2.3. Chapter Conclusion

In this chapter, an overview of the automatic control system using Arduino combined with the RC522 RFID module for card recognition was presented, with results displayed on an LCD screen and alert signals emitted via a buzzer. Initially, the chapter introduced the Arduino IDE software, highlighting its advantages, disadvantages, and typical applications in both daily life and industrial contexts. Next, the operating principle of the circuit was analyzed in detail, describing the process from when the RFID card is scanned to when the system processes and compares the

UID to decide whether to grant or deny access, while simultaneously activating the buzzer for alerts. Additionally, the chapter provided algorithm diagrams and data flowcharts to clearly illustrate the processing and information exchange between functional blocks. As a result, this chapter established a solid theoretical foundation for the design, programming, and deployment of the system in the subsequent chapters.

CHAPTER 3: SYSTEM IMPLEMENTATION AND EVALUATION

3.1. Chapter Introduction

In this chapter, the results of the system implementation, the circuit hardware, and related evaluations are presented.

3.2. Hardware implementation and performance results

Implementation of the printed circuit board and implementation results.

The components used on the circuit board.

Table 3.1: Components Used for the Circuit Board

No.	Component Name	Quantity
1	ESP32	1
2	Buzzer	1
3	LCD 16×2 Display	1
4	I2C Module	1
5	Servo Motor	2
6	Module RFID RC522 12.56MHz	2
7	RFID Card	
8	LM2596 Step-Down Converter Module	1
9	5V Relay Module	2
10	Raspberry pi 3	1
11	ICON Webcam Q-20	2

3.3. Results

3.3.1. Printed Circuit Board Results

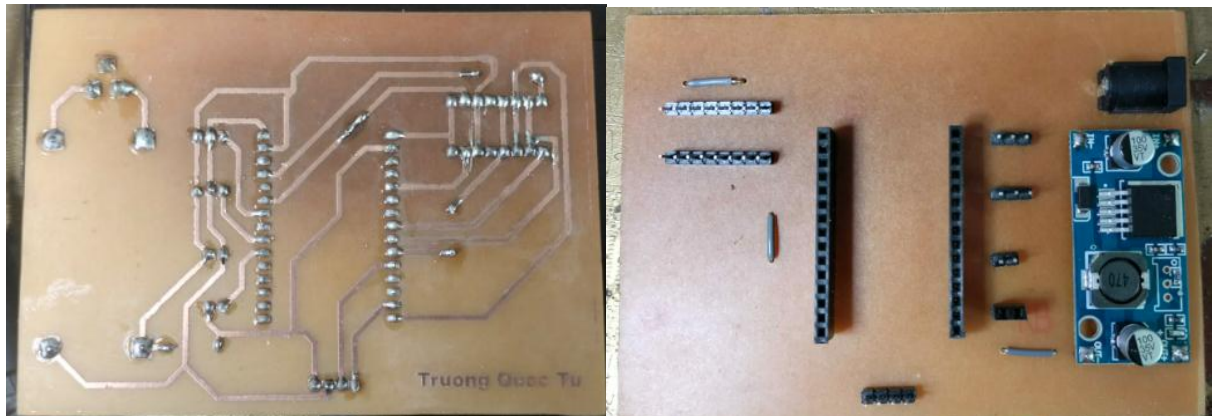


Figure 3.1: Printed Circuit Board Results

3.3.2. Results of Implementing the Parking Management System

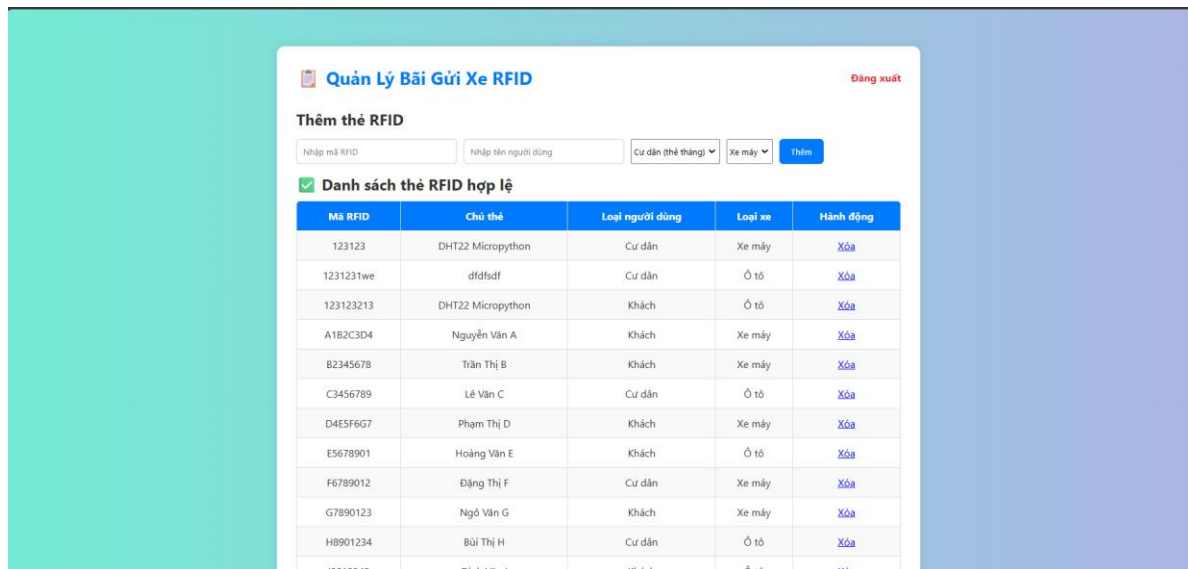


Figure 3.2: Website Interface and Displayed Information

Functions:

a. Valid and Invalid Card Identification

- When a user scans a card, the system checks whether the UID exists in the list of valid cards.
- If it exists, the system allows the gate to open.
- If it does not exist, an alert is triggered and the event is stored in the invalid_logs table.

b. Storage of User and Vehicle Information

In the valid card table, the following information is typically stored:

- Card UID
 - User name
 - User type: resident / visitor
 - Vehicle type: motorcycle / car
- => Enables the system to accurately apply appropriate fee structures based on user categories.

c. Classification of Usage Type

- Indicates the type of card:
 - + Monthly card (resident – no minutely charge applied)
 - + Single-use card (visitor – fee calculated based on parking duration)

3.3.3. Results of Implementing Entry and Exit History for the Parking System



L2345678	Vào → Ra	2025-07-07 09:00:00 → 2025-07-07 10:00:00	Khách	Xe máy	1	5,000
M3456789	Vào → Ra	2025-07-08 10:30:00 → 2025-07-08 13:30:00	Cư dân	Xe máy	3	0
N4567890	Vào → Ra	2025-07-09 07:50:00 → 2025-07-09 09:20:00	Khách	Ô tô	1.5	20,000
O5678901	Vào → Ra	2025-07-10 08:20:00 → 2025-07-10 09:30:00	Cư dân	Xe máy	1.17	0
P6789012	Vào → Ra	2025-07-11 07:30:00 → 2025-07-11 08:40:00	Khách	Xe máy	1.17	10,000
Q7890123	Vào → Ra	2025-07-12 10:10:00 → 2025-07-12 11:50:00	Cư dân	Xe máy	1.67	0

Xuất Excel

Figure 3.3: Entry and Exit History

Function:

The entry/exit history table (logs) stores all information related to the entry and exit process of each RFID card in the system. This is one of the most important tables, as it enables the system to monitor activity, calculate parking fees, and ensure transparent data management.

a. Recording Vehicle Entry and Exit Time

- Each time a user scans an RFID card:
 - + If there is no previous exit record, the system creates a new record => entry time.

- + When the card is scanned again for exit, the system updates the record => exit time.
- This allows the system to determine:
 - + Parking duration
 - + Entry and exit frequency of each vehicle

b. Basis for Parking Fee Calculation

- Based on:
 - + Entry time
 - + Exit time
 - + User type (resident / visitor)
 - + Vehicle type (car / motorcycle)
- The system calculates the fee as follows:

Visitors:

- + Motorcycle: 5,000 VND/minute
- + Car: 10,000 VND/minute

Residents: free of charge (monthly card)

- The calculated fee is stored in the fee column of the logs table.

3.3.4. Warning for Invalid Card Access

⚠ Cảnh báo: Truy cập bằng thẻ không hợp lệ		
Mã RFID	Thời gian	Hành động
1D9F1305	2025-07-22 10:39:01	Xóa
FAKECARD9	2025-07-15 12:15:10	Xóa
WRONGTAG2	2025-07-15 11:30:45	Xóa
INVALID001	2025-07-15 10:00:00	Xóa
XYZ67890	2025-07-14 09:45:20	Xóa

Figure 3.4: Invalid Access List

The invalid_logs table is used to store all instances of RFID card scans that are not included in the list of valid cards. This is an important component for enhancing system security and enabling administrators to easily detect abnormal situations.

a. Recording All Invalid Card Cases

- When a user scans an RFID card that does not exist in the valid_rfids table, the system will:

- + Not open the gate
- + Activate an alert via the LCD or buzzer
- + Record the event in the invalid_logs table
- The recorded information includes:
 - + Invalid card UID
 - + Scan timestamp
 - + Scan location (entry gate / exit gate)
 - + Image captured from the camera (if a Raspberry Pi is used)
 - + This allows the system to monitor and track all suspicious activities comprehensively.

b. Supporting Parking Lot Security

- The invalid_logs table helps to:
 - + Detect intentional intrusion attempts using unauthorized cards
 - + Monitor unauthorized card usage events
 - + Enhance security by associating license plate data with unknown cards through camera integration
 - + This is particularly important for residential complexes, offices, or high-traffic parking facilities.

c. Providing Data for Incident Handling by Administrators

- Administrators can review:
 - + The number of times an unauthorized card is scanned
 - + Who performed the scan (based on license plate recognition or captured images, if available)
 - + Whether the invalid scan was caused by a new resident, a malfunctioning card, or an unauthorized individual
- This supports appropriate actions such as:
 - + Reissuing cards
 - + Delete card
 - + Alerting security personnel when abnormal behavior is detected

d. Supporting Statistics and Reporting

- The invalid_logs table can be used to:
 - + Count invalid scans by day, week, or month
 - + Assess the overall security level of the system
 - + Identify peak periods of invalid scan occurrences
 - + Support system improvement or enhanced security measures

3.4. System Evaluation

The process of designing and developing the simulated parking management system utilizes RFID technology, with the ESP32 serving as the central control unit in combination with key components such as the RC522 RFID reader module, SG90 servo motor to simulate the gate barrier, LCD1602 display for status indication, and a buzzer for audible alerts. The circuit is designed with a simple structure, easy to assemble, operates stably in a simulated environment, and is suitable for educational and research purposes.

The control program is developed using the Arduino IDE with a clear and well-organized structure, enabling efficient processing of card identification, information display, and gate opening/closing control based on the card scanning status. The system demonstrates several notable advantages, including low cost, widely available components, ease of programming, and strong potential for expansion with additional features such as data storage, network connectivity, or the development of a remote management system.

However, the system still has certain limitations. It currently operates only at the simulation level and has not yet been deployed in a real-world environment. Card data are stored statically within the program and cannot be updated dynamically. The system lacks the capability to store access history or synchronize data over the Internet. Additionally, the use of a breadboard makes the circuit susceptible to loose connections and does not ensure long-term durability during extended operation.

These limitations provide a foundation for further improvement and development in subsequent stages, with the goal of moving toward practical deployment and a more intelligent and robust parking management solution.

CONCLUSION AND FUTURE DEVELOPMENT DIRECTIONS OF THE PROJECT

1. Conclusion

In the context of rapid technological advancement, the demand for applying automation systems to daily life is increasingly growing, particularly in the fields of transportation, vehicle management, and access control. With the objective of researching, simulating, and developing a parking management system using RFID technology, this project has been implemented with a practical-oriented approach, integrating both hardware and control software based on the Arduino Uno microcontroller.

Through the process of investigation, design, and experimentation, the project has achieved several notable results:

- Successfully designed a simulated smart parking system model using the RC522 RFID module for contactless card identification.
- Achieved stable automatic gate barrier opening and closing control using the SG90 servo motor based on card authentication results.
- Provided status alerts via a buzzer and displayed intuitive information through the LCD1602 screen, enabling users to easily recognize card scanning outcomes.
- All system operations are processed by the Arduino microcontroller, which offers flexible programmability and strong potential for future expansion.

Through the implementation of this project, the team had the opportunity to apply and strengthen knowledge in electrical and electronic engineering, microcontroller technology, embedded programming skills, and practical control signal processing. In particular, the project contributed to enhancing systems thinking, teamwork capability, and problem-solving skills throughout the development of the simulated product.

2. Limitations

Although certain achievements have been made, the project still has several limitations that need to be addressed in the next development stages:

- The system is limited to simulation and has not been implemented in practice with industrial devices such as real barriers, field sensors, etc.

- RFID card data is stored statically in the program (hard-coded).
- Security is basic: using the original UID, which can be easily cloned in an open environment if no additional encryption layer is applied.
- Lacks supplementary sensors such as distance sensors and vehicle count sensors to enhance automation capabilities.

3. Future Development Directions

To further improve the model and move toward practical application, the team proposes several future development directions as follows:

- Upgrade the system to a real-world model
 - + Integrate a real barrier instead of a simulated servo.
 - + Install the system at actual parking locations (schools, residential complexes, public parking lots).
 - + Use a stable power supply and appropriate interference-proof devices.
- IoT connectivity – remote management
 - + Provide alerts via email or phone if a fake card or unauthorized access is detected.
- Enhance security features
 - + Add a UID encryption layer or implement a two-factor authentication system (card + PIN or OTP).
 - + Integrate biometric sensors (fingerprint, face recognition) to verify the vehicle owner.
- Dynamic RFID card management
 - + Update, add, or remove cards via the LCD screen and push buttons.
 - + Link cards to specific user information: license plate, user name, phone number, etc.
- System scalability expansion
 - + Manage multiple vehicle lanes simultaneously.
 - + Connect multiple RFID stations into a centralized management system.
 - + Integrate vehicle-counting sensors and parking duration measurement to enable automatic fee calculation.

4 Conclusion

During the course of this project, the intelligent parking system using RFID combined with license plate recognition has been researched, designed, and implemented based on knowledge of embedded systems, IoT, programming, and databases. The project focuses on addressing the problem of managing vehicle entry and exit in parking lots automatically, accurately, and securely, aiming to overcome the limitations of traditional management methods.

The system is built with the ESP32 as the central processor, integrated with hardware components such as the RC522 RFID module, camera, servo motor, buzzer, display unit, and power circuit, alongside web-based management software. Core functionalities, including RFID card scanning, license plate recognition, gate control, parking time recording, fee calculation, and data storage, have been implemented and tested successfully.

Experimental results indicate that the system operates relatively stably, meeting the basic requirements set out. The combination of RFID and license plate recognition enhances reliability, reducing the risk of fraud or errors during vehicle parking. Meanwhile, data is stored and managed centrally, supporting effective statistics, monitoring, and operation of the parking lot.

Through this project, the implementers had the opportunity to apply and consolidate theoretical knowledge, while improving skills in system design, microcontroller programming, data processing, and hardware–software integration. Although the system still has some limitations in scalability and optimization, the achieved results demonstrate the project’s feasibility and practical application value, laying a foundation for future development and system refinement.

REFERENCES

- [1] Tran Van Long (2021), *Microcontrollers and Applications*, Science and Technology Publishing House.
- [2] Arduino Official Website, Available at: <https://www.arduino.cc> (Accessed: 10 July 2025).
- [3] RFID RC522 Module Technical Documentation, Available at: <https://components101.com/wireless/mfrc522-rfid-reader-module> (Accessed: 10 July 2025).
- [4] TowerPro SG90 Servo Motor Datasheet, Available at: <https://www.towerpro.com.tw/product/sg90-2> (Accessed: 10 July 2025).
- [5] TutorialsPoint, *Internet of Things (IoT) – Architecture*, Available at: https://www.tutorialspoint.com/internet_of_things/internet_of_things_architecture.htm (Accessed: 10 July 2025).
- [6] Anil K. Jain, Arun Ross, Karthik Nandakumar (2011), *Introduction to Biometrics*, Springer.
- [7] Joseph J. Carr (2018), *RFID Handbook: Fundamentals and Applications in Contactless Smart Cards and Identification*, 3rd Edition, Wiley.
- [8] Satish Mishra, Reza Malekian (2019), “Smart Parking System using IoT and Embedded Systems”, *Journal of Embedded Systems*, Vol. 12, No. 4, pp. 145–159.
- [9] Vinod Kumar, Rajeev Kumar (2020), “License Plate Recognition Systems: Techniques and Applications”, *International Journal of Computer Applications*, Vol. 176, No. 45, pp. 1–8.